

A multi-critical-path co-driven evolutionary algorithm addressing the dual-resource flexible job shop scheduling problem with offline operations and job priority constraints

Ziyu Zhang, Dongchen Qiu, Xinyu Li, Liang Gao, Qihao Liu, Yue Teng, Xuxia Zhang & Jun Wu

To cite this article: Ziyu Zhang, Dongchen Qiu, Xinyu Li, Liang Gao, Qihao Liu, Yue Teng, Xuxia Zhang & Jun Wu (2026) A multi-critical-path co-driven evolutionary algorithm addressing the dual-resource flexible job shop scheduling problem with offline operations and job priority constraints, International Journal of Production Research, 64:11, 4310-4340, DOI: [10.1080/00207543.2025.2601263](https://doi.org/10.1080/00207543.2025.2601263)

To link to this article: <https://doi.org/10.1080/00207543.2025.2601263>



Published online: 10 Dec 2025.



Submit your article to this journal [↗](#)



Article views: 280



View related articles [↗](#)



View Crossmark data [↗](#)



Citing articles: 4 View citing articles [↗](#)



A multi-critical-path co-driven evolutionary algorithm addressing the dual-resource flexible job shop scheduling problem with offline operations and job priority constraints

Ziyu Zhang^a, Dongchen Qiu^b, Xinyu Li^a, Liang Gao^a, Qihao Liu^a, Yue Teng^a, Xuxia Zhang^c and Jun Wu^c

^aNational Center of Technology Innovation for Intelligent Design and Numerical Control, School of Mechanical Science and Engineering, Huazhong University of Science and Technology, Wuhan, People's Republic of China; ^bChina Tobacco Jiangsu Industrial Co. Ltd., Nanjing, People's Republic of China; ^cZhejiang Wanfeng Technology Development Co. Ltd., Shaoxing, People's Republic of China

ABSTRACT

In the customised manufacturing of complex structural components, such as precision instruments and ships, jobs often have different priorities and involve a mix of online and offline operations. To address these challenges, this paper studies the dual-resource flexible job shop scheduling problem with special workers and job priority constraints (DRFJSP-OJP). Correspondingly, a mixed-integer linear programming (MILP) model is developed, and a multi-critical-path co-driven evolutionary algorithm (MCPEA) is proposed, which includes three key innovations. Firstly, a priority-driven three-layer segmented encoding and priority-based multi-segment active decoding scheme is designed. Secondly, a migration operator based on exemplar selection is introduced to accelerate the convergence. Finally, the global critical-path of the problem and local critical-paths with priorities are defined, then a problem-specific neighbourhood structure is designed. The experimental results indicate that the constructed MILP model can successfully solve small-scale problems. Meanwhile, MCPEA demonstrates superior overall performance than other methods, not only improving production efficiency but also ensuring the timely processing of high-priority jobs. Finally, MCPEA is applied to a real-world case from a complex structural component manufacturing enterprise. The optimised scheduling scheme shortens makespan by 49.60%, and decreases delay rate by 33.33%.

ARTICLE HISTORY

Received 9 July 2025

Accepted 27 November 2025

KEYWORDS

Flexible job shop scheduling; dual-resource; job priority; evolutionary algorithm; neighbourhood structure

1. Introduction

The production scheduling problem, as a core decision-making challenge in intelligent manufacturing systems, has long attracted attention from both academia and industry (Bakon et al. 2022; Y. Li, Li et al. 2025; Zhou et al. 2022). Among its variants, the flexible job shop scheduling problem (FJSP) shortens the production cycle by optimising operation sequence and machine assignment, providing a theoretical basis for enhancing production efficiency and bearing significant practical importance (Dauzère-Pérès et al. 2024; Li and Gao 2016; Xie, Gao et al. 2019; Yang et al. 2025). However, the traditional FJSP model focuses solely on machines as the core resource, overlooking the impact of workers' skill heterogeneity on production efficiency and quality (Destouet et al. 2023; Thürer, Stevenson, and Renna 2019). In real-world discrete manufacturing workshops, workers usually represent another crucial and often bottleneck resource (Goli, Tirkolaee, and Aydın 2021; Mraihi, Driss, and El-Haouzi 2024; Vahedi-Nouri et al. 2024; Wang, Hu,

and Gong 2018). To address it, researchers have proposed the dual-resource flexible job shop scheduling problem (DRFJSP), aiming to collaboratively optimise both the machine and worker resources (Thürer et al. 2020; Zhang, Wang, and Xu 2017; Zheng and Wang 2016). In contrast, this model offers a closer approximation to practical scenarios and provides valuable guidance for improving the feasibility and accuracy of production planning. As an important extension, DRFJSP has rapidly garnered research interest globally, with a growing body of related literature (e.g. Chen, Kuo, and Chien 2024; Gnanavelbabu, Caldeira, and Vaidyanathan 2021; X. Gong et al. 2018; Meng et al. 2019; Usman, Lu, and Gao 2024; C. Wang, Fan et al. 2025).

Nevertheless, most existing DRFJSP models contain a key assumption that all operations require simultaneous allocation of both machine and worker resources (e.g. Andrade-Pineda et al. 2020; G. Gong, Chiong et al. 2020; Han and Gong 2025; Q. Luo, Deng, Xie et al. 2023; C. Wang, Fan et al. 2025). Because these

studies generally consider that all operations must be processed on machines and cannot be separated from the machining line. However, this constraint exhibits obvious limitations in discrete manufacturing industries with complex structural components, such as precision instruments and ships, primarily because some offline operations in the process route are not considered. Taking the reference mirror base in high-precision measuring instruments as an example. Its manufacturing process not only involves the operations performed on machines such as lathes, milling machines, and planers, but also includes operations like cryogenic treatment, aging, and marking, which do not need to be completed on the machining line. These offline operations rely on fixed or dedicated workbenches, do not require the assignment of machine resources, but must be performed by workers with the corresponding skills (J. Li et al. 2025). Neglecting this characteristic leads to models that cannot accurately reflect the resource competition situation, leading to deviations in production cycle and imbalance in resource allocation. Therefore, it is necessary to incorporate the unique resource constraints of offline operations into the scheduling model to improve the utilisation of critical resources.

On the other hand, existing studies generally assume that all jobs possess the same priority (e.g. C. Luo et al. 2025; Ren et al. 2022; Shi et al. 2023; C. Wang, Wei et al. 2025; Z. Zhang, Li et al. 2025). However, in actual production environments, especially in order-driven discrete manufacturing, significant priority differences exist among jobs due to economic benefit and delivery urgency (Kim 2020; Sara 2025). For orders with larger profit margins or higher late delivery costs, enterprises will prioritise them as much as possible during scheduling (Chen et al. 2020; Ham 2017; Z. Zhu et al. 2022). Similarly, taking the structural components of precision instruments as our example, the reference mirror base, as a fundamental core component ensuring measurement accuracy, is assigned higher priority since any delivery delay would halt subsequent installation and debugging. On the contrary, the production of non-core structural components (such as the support base, equipment enclosure, and auxiliary sliding table) allows for a certain degree of flexibility and thus lower priority. Scheduling schemes that ignore job priorities, even if they theoretically achieve the optimal production efficiency, are likely to cause high-value jobs to be delayed due to waiting for low-value ones in practice, resulting in substantial economic losses (Li, Peng, and Wang 2024). From this perspective, developing scheduling methods that ensure high-priority jobs are processed as early as possible is an inevitable requirement for enhancing the practicality of production plans and realising value-driven production.

In conclusion, the insufficient consideration of offline operations and job priority differences significantly weakens the practical guiding value of existing DRFJSP theoretical models (Delgoshaei et al. 2023; Mlekusch and Hartl 2025). This obvious gap between theory and practice necessitates a more specific and systematic scheduling model and method that can reflect the complexity of the real-world manufacturing environment. Therefore, the starting point of this paper is to fill the gap in current research regarding offline operations and job priorities, so that the scheduling theory can be effectively applied to complex manufacturing fields such as precision instruments and ships. By focussing on these two critical characteristics, this paper investigates a dual-resource flexible scheduling problem that better aligns with operation types and production urgency, thereby developing an efficient scheduling method that satisfies different resource spatio-temporal constraints and prioritises high-priority tasks.

Driven by this motivation, the DRFJSP with offline operations and job priority constraints (DRFJSP-OJP) is defined in this work. To address this issue, a mixed integer linear programming (MILP) model is developed, then a multi-critical-path co-driven evolutionary algorithm is proposed, referred to as MCPEA. In this algorithm, a hierarchical multi-segment encoding and decoding scheme is presented for the special constraints brought by offline operations and job priorities. Based on this, the initialisation method, migration strategy and mutation operator based on problem characteristics are proposed, and a priority-driven neighbourhood structure is designed. Experimental results demonstrate that the proposed algorithm can obtain high-quality solutions, with overall performance significantly surpassing that of other advanced scheduling methods. Finally, the MCPEA is applied to a real case of one complex structural component manufacturing enterprise, resulting in a 49.60% reduction in the original makespan and a 33.33% decrease in the delay rate. The main contributions and innovations can be summarised as follows:

- (1) The MILP for DRFJSP-OJP is developed, where the skill diversity of workers is represented by different constraint sets based on the types of operations, achieving the structured expression of the problem.
- (2) A three-layer segmented encoding scheme that aligns with the characteristics of offline operations and job priorities is designed, and a priority-based multi-segment active decoding method is proposed. This framework ensures the resource legality and constraint satisfaction of all solutions.
- (3) The unique global critical-path of the problem is defined, along with multiple local critical-paths of

different job priorities. By extracting common critical operations, a problem-specific neighbourhood structure that can effectively explore the solution space is designed.

The rest of this paper is organised as follows. The literature review is analysed in Section 2. In Section 3, DRFJSP-OJP is described in detail, and the MILP model is developed. Then, the algorithm is systematically proposed in Section 4. Comparison experiments are provided in Section 5. Further, Section 6 conducts a study on an actual case from a complex structure component production workshop, along with specific management insights. Finally, the conclusion and future work are given in Section 7.

2. Literature review

2.1. DRFJSP variants

FJSP serves as a fundamental model for complex manufacturing environments, extending the classical job shop scheduling problem (JSP) (Hajariwala, Patil, and Patil 2025; Huang, Zhao, and Han 2022; Xiong et al. 2022) by allowing operations to be processed on any machine within a given set of compatible machines. Early foundational work by Brucker and Schlie (1990) established the computational complexity of FJSP, while later research focussed on developing efficient solution methods, such as mathematical programming (Lim and Moon 2023), meta-heuristic (Han et al. 2024), and reinforcement learning (Lei et al. 2022). In recent years, recognising the limitations of the standard FJSP model in capturing real-world complexities, researchers have progressively incorporated various practical constraints. Such as the random processing time with probability distribution (Caldeira and Gnanavelbabu 2021), transportation resource constraints (Pan et al. 2022), sequence-dependent setup time (Z. Zhang, Fu et al. 2024), lot streaming (Yang et al. 2025), multi-machine collaboration (Huang, Li, and Gao 2025), and so on. However, these variants predominantly focus on machine allocation, overlooking the critical role of labor in real-world production systems. This led to the development of the DRFJSP, which explicitly incorporated the requirement for worker resources into models.

Thürer, Stevenson, and Renna (2019) investigated the performance of workload control in dual-resource constrained high-variety shops, considering the interchangeable labor. Andrade-Pineda et al. (2020) studied the DRFJSP focussing on product customisation and on-time delivery, where worker expertise influences both eligibility and processing time. H. Zhu et al. (2020) took

workers into account and considered the effects of their learning abilities on the processing time and energy consumption. On this basis, Peng et al. (2022) developed a multi-objective model constrained by job transportation time and worker learning effect. Then, the DRFJSP with time window constraints was defined by Frihat, Hadj-Alouane, and Sadfi (2022), and was solved using MILP and constraint programming (CP) models. Li, Wang, and Peng (2022) first discussed the job priority constraints in the DRFJSP, although they did not propose a specific optimisation method. He, Tang, and Luan (2022) developed a MILP model for DRFJSP aimed at minimising both makespan and total tardiness, effectively capturing the additional constraints arising from worker flexibility. Considering that when dealing with complex operations, machines may require multiple workers, Q. Luo, Deng, Xie et al. (2023) proposed the DRFJSP with the flexibility of worker cooperation. Usman, Lu, and Gao (2024) noticed that excessive work for workers might lead to reduced productivity, so studied the DRFJSP with limited flexible workers. Tang et al. (2024) extended the dual-resource flexibility to the distributed scenario and studied the significant impact of operation sequencing flexibility on production efficiency. Furthermore, the problem variant with worker shift arrangement was addressed, considering constraints such as job priority and resource unavailability (Li, Peng, and Wang 2024). Vahedi-Nouri et al. (2024) explored an integrated production scheduling and workforce planning problem in the reconfigurable manufacturing system (RMS) benefiting from reconfigurable machines and human-machine collaboration for the first time. Recently, J. Li et al. (2025) discussed the DRFJSP with multitasking workers who can perform multiple operations simultaneously. Z. Zhang, Li et al. (2025) studied a variant in the production line reconfiguration scenario, which further expanded the scope of dual-resource scheduling.

2.2. Scheduling algorithms

Initial approaches for scheduling are primarily dominated by heuristic rules (Kaban, Othman, and Rohmah 2012; Wang 2005) and operational research (OR) techniques (Fattahi et al. 2014; Lopes and de Carvalho 2007; Shahvari, Logendran, and Tavana 2022). While heuristic rules can offer computational simplicity and rapid solution time, they typically struggle to get the optimal scheduling solution. Conversely, OR methods can guarantee the optimal solution for the scheduling model, but, their significant computational overhead renders them impractical for large-scale problems. In recent years, swarm intelligence optimisation algorithms have emerged as a powerful alternative (Gao et al. 2019). Their

demonstrated effectiveness in handling the inherent complexity, scale, uncertainty, and nonlinearity characteristic of real-world engineering challenges (K. Wang et al. 2023; Z. Zhang, Gao et al. 2023; Zuo and Gao 2025) has led to their widespread adoption in shop scheduling research (Li and Wang 2022; Neufeld, Schulz, and Buscher 2023; C. Wang, Wei et al. 2025).

Gao et al. (2016) proposed a discrete harmony search algorithm for solving FJSP. Ahmadi-Javid and Hooshangi-Tabrizi (2017) developed an anarchic society optimisation (ASO) algorithm to solve the problem of scheduling machines, transporters, and employee timetabling in a job-shop environment. Huang, Guan, and Yang (2018) combined the genetic algorithm (GA) with the particle swarm optimisation (PSO), and proposed a hybrid genetic PSO algorithm based on teaching and learning to solve multi-objective FJSP. After that, Chen et al. (2020) proposed a self-learning GA. Mahmoodjanloo et al. (2020) introduced an adaptive differential evolution (DE) algorithm designed to minimise makespan for effectively addressing the FJSP involving reconfigurable machine tools. Müller and Kress (2022) proposed a meta-heuristic based on disjunctive graph encoding to solve the DRFJSP, and designed a local search based on the critical-path. Similarly, Lou et al. (2022) employed a neighbourhood structure based on the critical-path to develop a multi-objective evolutionary algorithm, aiming to address the DRFJSP that incorporates the worker learning-forgetting effects. Z. Zhu et al. (2022) proposed a shuffled cellular evolutionary grey wolf optimiser (GWO), which ensured the satisfaction of job priority constraints by binary sort tree-based repair mechanism. Wang, Wang, and Xiu (2023) proposed a cooperative memetic algorithm (MA) to tackle the energy-aware distributed welding shop scheduling problem. Subsequently, Li, Li, and Gao (2024) studied the distributed heterogeneous permutation flow-shop scheduling problem, and proposed an iterative greedy algorithm with acceleration of job allocation probability. Huang, Gong, and Lu (2024) addressed the type-2 green fuzzy JSP by proposing a MA that incorporates a hierarchical heuristic neighbourhood search. Yu et al. (2024) studied the energy-efficient distributed heterogeneous welding shop scheduling problem, and designed a self-learning artificial bee colony (ABC) algorithm. Of late, Xu, Wang et al. (2025) designed a chaotic encoding scheme into the quantum PSO, and conducted a comprehensive study on the application of this method in solving FJSP. Deng et al. (2025) further developed a knowledge-driven MA that incorporated a two-stage mutation mechanism to encourage the generation of new scheduling solutions. To address the scheduling challenges posed by the worker cooperation flexibility, Han

and Gong (2025) introduced the algorithm based on non-dominated levels and defined a critical path-driven local search operator.

2.3. Research gap

We summarise the previous research in Table 1. According to this table, the DRFJSP considering machine and worker resource constraints is a highly valuable research topic. Currently, scholars have incorporated several practical production constraints into this scheduling model, such as the learning-forgetting effect, sequencing flexibility, outsourcing constraints, transportation constraints, and machine speed. At the same time, many meta-heuristic methods for solving DRFJSP are developed, primarily using critical-path-based neighbourhood structures to implement local search. However, through further analysis, we can identify the following key limitations in the existing literature when it is applied to real industrial scenarios.

Firstly, the presence and impact of offline operations have not been systematically incorporated into the scheduling model. The vast majority of studies assume that all operations must be completed on the machining line, meaning that both machine and worker resources must be allocated simultaneously. Secondly, the inherent job priority constraint within the production system is often overlooked or treated simplistically. Existing studies (e.g. Gnanavelbabu, Caldeira, and Vaidyanathan 2021; G. Gong, Deng, Gong et al. 2021; Han and Gong 2025; Hongyu and Xiuli 2021; J. Li et al. 2025) frequently assume all jobs are equally important, failing to fully capture the widespread need for the priority-driven scheduling decision.

In solution methods, the existing algorithms do not effectively address the specific constraints posed by offline operations and job priorities. Traditional encoding schemes often lead to chromosomal decoding errors and lack priority-driven search operators. On the other hand, existing neighbourhood structures are primarily designed for the single resource (Liu, Yao, and Zhu 2021; Xie, Li et al. 2023; Xu, Zhang et al. 2024; C. Zhang et al. 2007), lacking consideration of the collaboration between machine and worker resources. At the same time, current neighbourhood search methods mainly rely on a single critical-path, whereas jobs with different priorities may have multiple corresponding local critical-paths. Although Tang et al. (2024) or Fan et al. (2021) proposed a neighbourhood structure based on double or multiple critical-paths, these methods are not directly applicable to problems with job priorities. To sum up, an effective scheduling algorithm should be designed specifically to deal with the specific characteristics of

Table 1. The summarisation of existing research.

Literature	Workshop type	Production constraints	Encoding and decoding	Neighborhood structure
Zheng and Wang (2016)	FJSP	Worker flexibility	Two-layer encoding, Semi-active decoding	×
Ahmadi-Javid and Hooshangi-Tabrizi (2017)	JSP	Worker shift, Transportation	Random key, Semi-active decoding	×
Zhang, Wang, and Xu (2017)	FJSP	Worker flexibility	Two-layer encoding, Semi-active decoding	Critical-path
G. Gong, Gong, Liu et al. (2018)	FJSP	Worker flexibility	Three-layer encoding, Active decoding	×
X. Gong et al. (2018)	FJSP	Worker flexibility	Three-layer encoding, Active decoding	Critical-path
Andrade-Pineda et al. (2020)	FJSP	Worker flexibility	Disjunctive graph	×
G. Gong, Chiong et al. (2020)	FJSP	Worker flexibility	Three-layer encoding, Active decoding	Critical-path
H. Zhu et al. (2020)	Low carbon FJSP	Worker flexibility, Worker learning	Three-layer encoding, Active decoding	Critical-block
Chen et al. (2020)	Open shop	Job priority	Precedence graph	×
Gnanavelbabu, Caldeira, and Vaidyanathan (2021)	Stochastic FJSP	Worker flexibility	Two-layer encoding, Semi-active decoding	×
Liu, Yao, and Zhu (2021)	Green FJSP	Worker flexibility	Three-layer encoding, Active decoding	×
Tan et al. (2021)	FJSP	Worker fatigue, Transfer time	Three-layer encoding, Active decoding	Critical-path
Peng et al. (2022)	FJSP	Worker learning effect	Three-stage encoding, Semi-active decoding	Critical-path
Müller and Kress (2022)	FJSP	Worker flexibility	Disjunctive graph	Critical-path
Q. Luo, Deng, Gong et al. (2022)	Distributed FJSP	Worker flexibility	Two-layer encoding, Semi-active decoding	Critical-path
Li, Wang, and Peng (2022)	FJSP	Worker flexibility, Job priority, Outsourcing	Random key, Active decoding	×
Lou et al. (2022)	FJSP	Worker flexibility	Three-layer encoding, Active decoding	Critical-path
Z. Zhu et al. (2022)	FJSP	Job priority	Three-layer encoding, Semi-active decoding	×
He, Tang, and Luan (2022)	FJSP	Worker flexibility	Three-layer encoding, Semi-active decoding	×
Shi et al. (2023)	FJSP	Worker flexibility	Two-layer encoding, Semi-active decoding	×
Q. Luo, Deng, Xie et al. (2023)	FJSP	Worker cooperation flexibility	Two-layer encoding, Active decoding	Critical-path
Vital-Soto, Baki, and Azab (2023)	FJSP	Worker flexibility, Sequencing flexibility	Three-layer encoding, Active decoding	×
Y. Li, Chen et al. (2023)	JSP	Worker flexibility, Machine layout, Transportation	Three-layer encoding, Semi-active decoding	×
Luo, Gong, and Lu (2024)	FJSP	Machine speed	Three-layer encoding, Semi-active decoding	Critical-path
Tang et al. (2024)	Distributed FJSP	Worker flexibility, Sequencing flexibility	Five-layer encoding, Semi-active decoding	Double critical-paths
Li, Peng, and Wang (2024)	FJSP	Worker shift, Job priority	Random key, Active decoding	×
Huang, Gong, and Lu (2024)	Fuzzy FJSP	Type-2 fuzzy processing time	Two-layer encoding, Semi-active decoding	Critical-path
Han et al. (2024)	FJSP	AGV allocation	Two-layer encoding, Semi-active decoding	×
J. Li et al. (2025)	FJSP	Multitasking worker	Two-layer encoding, Semi-active decoding	Critical-path
Z. Zhang, Li et al. (2025)	FJSP	Worker flexibility, Auxiliary resource	Three-layer encoding, Active decoding	Critical-path
Deng et al. (2025)	Distributed FJSP	Machine endurance	Three-layer encoding, Semi-active decoding	Critical-path
Han and Gong (2025)	FJSP	Worker cooperation flexibility, learning-forgetting effect	Three-layer encoding, Active decoding	Critical-path
This work	FJSP	Worker flexibility, Offline operation, Job priority	Three-layer segmented encoding, Multi-segment active decoding	Multiple critical-paths

DRFJSP-OJP. Therefore, this work has certain theoretical value and guiding significance.

3. Problem formulation

3.1. Problem description

The DRFJSP-OJP is described as follows: n jobs with different priorities need to be processed on m machines by w workers, where each job J_i consists of h_i operations. Among them, the operations belonging to the same job have a strict processing sequence, and higher-priority jobs should be processed as early as possible. Each operation O_{ij} can be one of two types: online operation that must be processed on one machine selected from its optional machine set Ω_{ij} and one worker assigned from the optional worker set ψ_k of that machine, or offline operation that not requires the machine resource but must be performed by one worker with the required skill.

For online operations, workers must collaborate with the machines on the machining line, so they are in online processing mode. On the contrary, the workers are in offline processing mode because the offline operations are not carried out on the machining line. Then, workers who can perform both types of operations are defined as the dual-mode worker.

It can be found, solving the DRFJSP-OJP needs to optimise machine assignment, worker selection and operation sequence simultaneously. The objective is to minimise the maximum completion time while ensuring that high-priority tasks are processed as early as possible. Moreover, other constraints and related assumptions considered are as follows.

- Each machine can process only one job at a time.
- Each worker can only collaborate one machine or perform one task at a time.

Table 2. The specific processing information of each job in the example.

Jobs	Priority	Operations	Type	Processing time				
				M_1	M_2	M_3	W_1	W_3
J_1	2	O_{11}	1	12	–	13	–	–
		O_{12}	1	16	14	–	–	–
J_2	1	O_{21}	0	–	–	–	16	16
		O_{22}	1	–	18	17	–	–
J_3	1	O_{23}	1	14	12	–	–	–
		O_{31}	0	–	–	–	–	22
		O_{32}	1	20	–	18	–	–
J_4	2	O_{33}	1	21	19	–	–	–
		O_{41}	1	–	26	24	–	–
		O_{42}	1	15	–	15	–	–
J_5	1	O_{43}	1	14	13	–	–	–
		O_{51}	0	–	–	–	18	18
		O_{52}	1	13	–	11	–	–
J_6	2	O_{61}	1	–	17	19	–	–
		O_{62}	1	23	–	23	–	–
		O_{43}	0	–	–	–	24	24
J_7	2	O_{71}	1	–	22	20	–	–
		O_{72}	1	21	–	24	–	–
		O_{73}	0	–	–	–	20	–

- Operation O_{ij} cannot be processed unless its preceding operation $O_{i,j-1}$ has completed.
- Once an operation begins to be processed, no interruption is allowed.
- The transfer time of jobs or workers between machines is ignored.

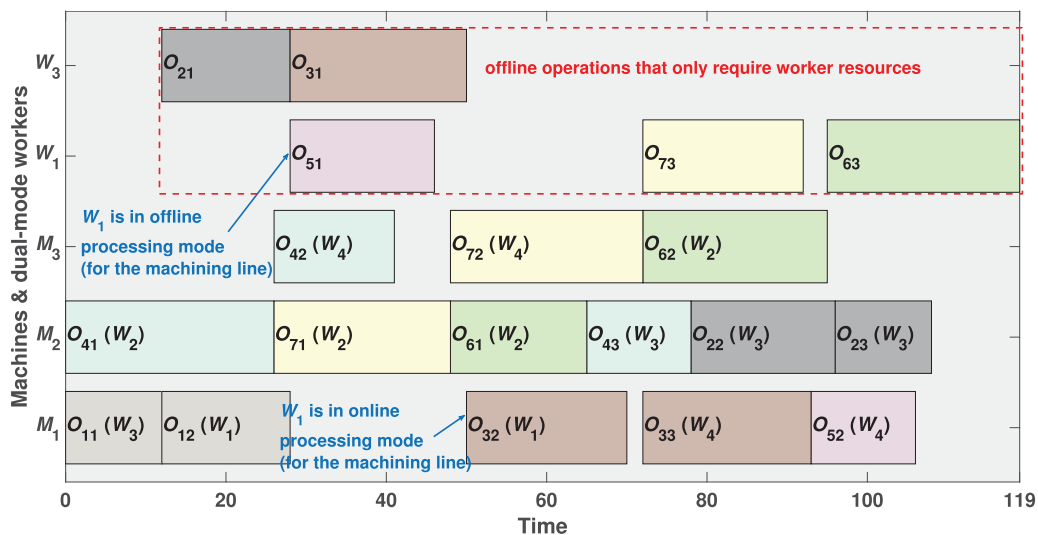
To clarify the relatively novel research issue of DRFJSP-OJP, a simple example is presented, as shown in Table 2. In this example, there are seven jobs $\{J_1, J_2, \dots, J_7\}$ to be processed in the workshop with three machines $\{M_1, M_2, M_3\}$ and four workers $\{W_1, W_2, W_3, W_4\}$. Among these, two workers $\{W_1, W_3\}$ are dual-mode workers, meaning they can perform both online and offline operations. The optional worker sets for the three

machines $\{M_1, M_2, M_3\}$ are $\{W_1, W_3, W_4\}$, $\{W_2, W_3\}$ and $\{W_1, W_2, W_4\}$, respectively. Where, ‘Type’ indicates the processing type of the corresponding operation. ‘1’ means an online operation that requires machine processing, and ‘0’ represents an offline operation. Then, ‘–’ means that this operation cannot be processed by that machine or worker.

Figure 1 illustrates the gantt chart of a feasible solution for this example problem. It can be observed that the processing of an operation can only begin when both the machine and worker are idle. Taking the operation O_{33} as an instance, after processing the O_{32} , the machine M_1 assigned to O_{33} is idle. However, the worker W_4 assigned to M_1 is operating on M_3 , so O_{33} cannot start immediately. Meanwhile, it can be found that the high-priority jobs $\{J_1, J_4, J_6, J_7\}$ are processed as early as possible, and their completion times are generally smaller than those of other jobs (e.g. J_2, J_3 , etc.). Furthermore, Figure 2 presents the gantt chart from the workers’ perspective corresponding to the same feasible solution. It can be observed that the dual-mode worker needs to leave the machining line when performing offline operations. For instance, W_3 is in offline mode when processing O_{31} , and then transfers to the machining line to process O_{43} .

3.2. Mathematical model

Through the abstraction of practical scenarios into mathematical formulations, optimisation objectives and constraints can be precisely defined (Seifi, Schulze, and Zimmermann 2021). Subsequently, a MILP model is formulated to capture the characteristics of the DRFJSP-OJP, and its notation description is shown in Table 3.

**Figure 1.** The gantt chart of a feasible solution for the example problem (machines' perspective).

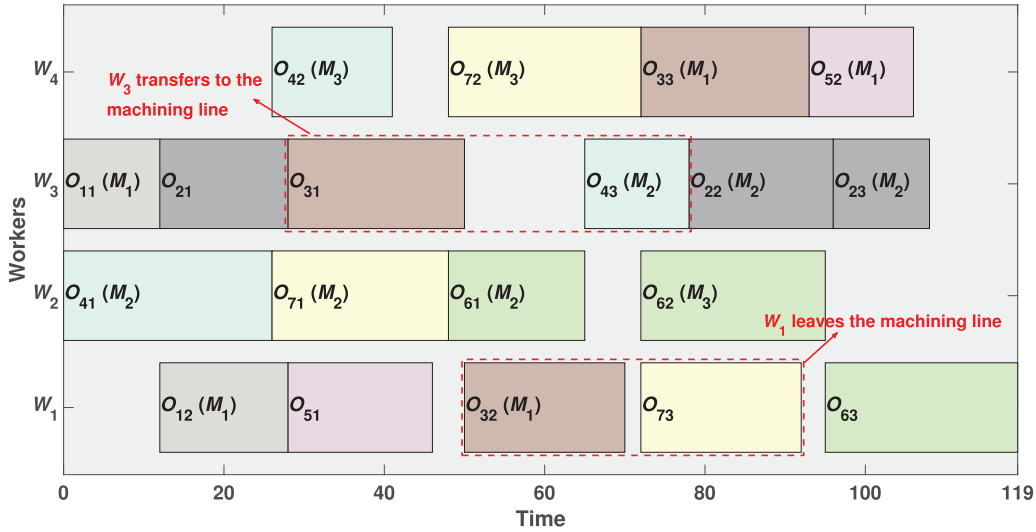


Figure 2. The gantt chart with the workers' perspective corresponding to the same feasible solution.

Objective function:

$$\min f = C_{\max} + \sum_{i=1}^n (p_i \cdot (C_i - r_i)) \quad (1)$$

The job priority is an open constraint without a corresponding constraint expression. To address it, we introduced a penalty mechanism into the objective function. By imposing penalties on the completion time, the consideration for processing high-priority jobs as soon as possible is directly reflected in the model.

Constraints:

$$\sum_{k \in \Omega_{ij}} X_{ijk} + \sum_{e \in \Phi_{ij}} Y_{ije} = 1 \quad (i = 1, 2, \dots, n; j = 1, 2, \dots, h_i) \quad (2)$$

if $N_{ij} = 1$

$$\sum_{k \in \Omega_{ij}} X_{ijk} = \sum_{e \in \Phi_{ij}} Z_{ijke} \quad (i = 1, 2, \dots, n; j = 1, 2, \dots, h_i) \quad (3)$$

$$S_{i1} \geq r_i \quad (i = 1, 2, \dots, n) \quad (4)$$

$$S_{ij} + \left(\sum_{k \in \Omega_{ij}} X_{ijk} p_{ijk} + \sum_{e \in \Phi_{ij}} Y_{ije} p_{ije} \right) \leq S_{i,j+1} \quad (i = 1, 2, \dots, n; j = 1, \dots, h_i - 1) \quad (5)$$

$$S_{i,h_i} + \left(\sum_{k \in \Omega_{i,h_i}} X_{i,h_i,k} p_{i,h_i,k} + \sum_{e \in \Phi_{i,h_i}} Y_{i,h_i,e} p_{i,h_i,e} \right) \leq C_{\max}, \quad i = 1, 2, \dots, n \quad (6)$$

if $N_{ij}, N_{i'j'} = 1$

$$\begin{cases} S_{ij} + p_{ijk} \leq S_{i'j'} + L(3 - A_{ijj'} - X_{ijk} - X_{i'j'k}) \\ S_{i'j'} + p_{i'j'k} \leq S_{ij} + L(2 + A_{ijj'} - X_{ijk} - X_{i'j'k}) \\ i, i' = 1, 2, \dots, n \mid i \neq i'; j = 1, 2, \dots, h_i; \\ j' = 1, 2, \dots, h_{i'}; k \in \Omega_{ij} \cap \Omega_{i'j'} \end{cases} \quad (7)$$

if $N_{ij}, N_{i'j'} = 0$

$$\begin{cases} S_{ij} + p_{ije} \leq S_{i'j'} + L(3 - B_{ijj'} - Y_{ije} - Y_{i'j'e}) \\ S_{i'j'} + p_{i'j'e} \leq S_{ij} + L(2 + B_{ijj'} - Y_{ije} - Y_{i'j'e}) \\ i, i' = 1, 2, \dots, n \mid i \neq i'; j = 1, 2, \dots, h_i; \\ j' = 1, 2, \dots, h_{i'}; e \in \Phi_{ij} \cap \Phi_{i'j'} \end{cases} \quad (8)$$

if $N_{ij}, N_{i'j'} = 1$

$$\begin{cases} S_{ij} + p_{ijk} \leq S_{i'j'} + L(5 - C_{ijj'} - X_{ijk} - X_{i'j'k'} - Z_{ijke} - Z_{i'j'k'e}) \\ S_{i'j'} + p_{i'j'k'} \leq S_{ij} + L(4 + C_{ijj'} - X_{ijk} - X_{i'j'k'} - Z_{ijke} - Z_{i'j'k'e}) \\ i, i' = 1, 2, \dots, n \mid i \neq i'; j = 1, 2, \dots, h_i; \\ j' = 1, 2, \dots, h_{i'}; k \in \Omega_{ij}; k' \in \Omega_{i'j'}; e \in \Psi_k \cap \Psi_{k'} \end{cases} \quad (9)$$

if $N_{ij} + N_{i'j'} = 1$

$$\begin{cases} S_{ij} + p_{ije} \leq S_{i'j'} + L(4 - D_{ijj'} - Y_{ije} - X_{i'j'k} - Z_{i'j'ke}) \\ S_{i'j'} + p_{i'j'k} \leq S_{ij} + L(3 + D_{ijj'} - Y_{ije} - X_{i'j'k} - Z_{i'j'ke}) \\ i, i' = 1, 2, \dots, n \mid i \neq i'; j = 1, 2, \dots, h_i; \\ j' = 1, 2, \dots, h_{i'}; e \in \Phi_{ij} \cap \Psi_k \end{cases} \quad (10)$$

Eq. (2) is the resource allocation constraint. for an online operation, only one machine can be assigned to it, while for an offline operation, only one worker can be selected. Then, the meaning of Eq. (3) is that if the machine assigned to it is M_k , then only one worker is definitely

Table 3. Description of notations of the mathematical model.

Notation	Description	
Indices	i, i'	Index of jobs, $i, i' = 1, 2, \dots, n$
	j, j'	Index of operations, $j, j' = 1, 2, \dots, h_i$
	k, k'	Index of machines, $k, k' = 1, 2, \dots, m$
	e, e'	Index of workers, $e, e' = 1, 2, \dots, w$
Parameters	n	Total number of jobs
	m	Total number of machines
	w	Total number of workers
	h_i	Total number of operations of J_i
	r_i	The release time of J_i
	p_i	The priority weight of J_i
	O_{ij}	The j -th operation of J_i
	Ω_{ij}	The optional machine set of the online operation O_{ij}
	Φ_{ij}	The optional worker set of the offline operation O_{ij}
	ψ_k	The assignable worker set of M_k
	p_{ijk}	The processing time of O_{ij} on M_k
	p_{ije}	The time for worker W_e to independently process O_{ij}
	N_{ij}	$= \begin{cases} 1, & \text{if } O_{ij} \text{ is an online operation} \\ 0, & \text{else} \end{cases}$
	L	A large enough positive number
Decision variables	X_{ijk}	$= \begin{cases} 1, & \text{if the online operation } O_{ij} \\ & \text{is processed on } M_k \\ 0, & \text{else} \end{cases}$
	Y_{ije}	$= \begin{cases} 1, & \text{if the offline operation } O_{ij} \\ & \text{is processed by } W_e \\ 0, & \text{else} \end{cases}$
	Z_{ijke}	$= \begin{cases} 1, & \text{if } M_k \text{ of the online operation } O_{ij} \\ & \text{is operated by } W_e \\ 0, & \text{else} \end{cases}$
	$A_{iji'j'}$	$= \begin{cases} 1, & \text{if } O_{ij} \text{ is processed before } O_{i'j'} \text{ on } M_k \\ 0, & \text{else} \end{cases}$
	$B_{iji'j'}$	$= \begin{cases} 1, & \text{if } W_e \text{ independently processes } O_{ij} \\ & \text{before } O_{i'j'} \\ 0, & \text{else} \end{cases}$
	$C_{iji'j'}$	$= \begin{cases} 1, & \text{if } W_e \text{ operates the machine for } O_{ij} \\ & \text{before the machine for } O_{i'j'} \\ 0, & \text{else} \end{cases}$
	$D_{iji'j'}$	$= \begin{cases} 1, & \text{if } W_e \text{ independently processes } O_{ij} \\ & \text{before operating } M_k \text{ to process } O_{i'j'} \\ 0, & \text{else} \end{cases}$
	S_{ij}	The start time of O_{ij}
	$C_{\max}$	The maximum completion time of all jobs

assigned to M_k . Eq. (4) indicates that each job can only be processed after it has been released. Eq. (5) is the operation sequence constraint: the end time of O_{ij} cannot exceed the start time of $O_{i,j+1}$. And Eq. (6) is the makespan constraints, that is the completion time of the last operation cannot exceed the $C_{\max}$. Subsequently, Eq. (7) represents the constraint set when processing online operations O_{ij} and $O_{i'j'}$ on the same machine M_k . Eq. (8) represents the constraint set when the same worker W_e independently processing offline operations O_{ij} and $O_{i'j'}$. While Eq. (9) represents the constraint set when the same worker W_e operates the machines M_k for O_{ij} and $M_{k'}$ for $O_{i'j'}$. Finally, Eq. (10) symbolises the constraint set when the same worker W_e independently processing O_{ij} and operating M_k for $O_{i'j'}$.

The logical correctness of these paired constraints (6) to (9) is ensured by the standard big- L method. The

binary variables A , B , C , and D are introduced to model the sequencing decisions between operations competing for the same resource: $A_{iji'j'}$ for the processing sequence on the same machine, $B_{iji'j'}$ for the independent processing sequence by the same worker, $C_{iji'j'}$ for the operating sequence of the same worker on different machines, and $D_{iji'j'}$ for the sequence between a worker's independent processing and machine operation tasks. For any given pair of operations, when the corresponding binary variable equals 1, the associated constraint is activated to enforce the sequence relationship. While it equals 0, the term multiplied by L becomes dominant and the constraint is deactivated. The value of L is a sufficiently large positive constant, thus guaranteeing the model's linearity.

4. Proposed algorithm

4.1. Framework of MCPEA

To overcome the limitations of existing methods in handling the job priority constraints and the dual-resource coupling relationship in DRFJSP-OJP, this section proposes the MCPEA. The algorithm features a priority-driven three-layer segmented encoding and active decoding to ensure solution feasibility. Meanwhile, it designs structured evolutionary operators that respect job priority constraints, and presents a problem-specific neighbourhood search method. Notably, MCPEA employs the migration operator from biogeography-based optimisation (BBO) (Simon 2008; Z. Zhang, Gao et al. 2023) instead of traditional crossover operator. This design better preserves the priority-based segmented structure, utilises the cosine migration model for quality-guided information exchange without additional parameters, and reduces the risk of disrupting superior solutions while enhancing the population diversity.

The following subsections detail the technical components: Subsection 4.2 presents the solution representation method, Subsection 4.3 describes population initialisation, Subsections 4.4 and 4.5 detail the migration and mutation operators respectively, while Subsection 4.6 explains the neighbourhood search method. The overall procedure is outlined in Algorithm 1, with specific parameter configurations including population size Ps , mutation probability Mp , and local search probability Lp provided in the experiments. In addition, although the MILP model introduces a penalty term to reflect priority constraints, the algorithm uses makespan as the fitness function for clear evaluation and comparison of solution efficiency. The handling of job priority constraints is embedded in the customised encoding and decoding scheme, and search strategies.

Algorithm 1 The pseudo code of MCPEA

```

1: Parameter setting, including population size ( $Ps$ ), mutation probability( $Mp$ ), and local search probability ( $Lp$ )
2: For ( $i = 1$  to  $Ps$ ) do % Subsection 4.3
3:   Hybrid heuristic initialization
4:   Calculate the objective function value  $f(x_i)$  for  $x_i$ 
5: End for
6: Determine the ranking of all individuals in the population
7: Calculate the species number ( $S_i$ ) of each individual
8: Calculate the immigration rate ( $\lambda_i$ ) of each individual
9: While (termination condition is not met) do
10:   For ( $i = 1$  to  $Ps$ ) do % Subsection 4.4
11:     If ( $rand(0, 1) \leq \lambda_i$ ) do
12:       Use Equation (16) to select a example exemplar  $x_j$ 
13:       Perform the migration operator
14:     End if
15:     If ( $rand(0, 1) \leq Mp$ ) do % Subsection 4.5
16:       Perform the mutation operator
17:     End If
18:   End for
19:   For ( $i = 1$  to  $Ps$ ) do
20:     Calculate the new objective function value  $f(x'_i)$  for  $x'_i$ 
21:     If ( $f(x'_i) < f(x_i)$ ) do
22:        $x'_i = x_i; f(x_i) = f(x'_i)$ 
23:     End if
24:   End for
25:   For ( $i = 1$  to the top 30% in  $Ps$ ) do % Subsection 4.6
26:     If  $rand(0,1) \leq Lp$ 
27:       Determine multiple critical-paths by Algorithm 2
28:       Extract common critical operations and perform the neighborhood search by Algorithm 3
29:       Calculate the new objective function value  $f(x'_i)$  for  $x'_i$ 
30:       If ( $f(x'_i) < f(x_i)$ ) do
31:          $x'_i = x_i; f(x_i) = f(x'_i)$ 
32:       End if
33:     End if
34:   End for
35:   Determine the ranking of all individuals in the population
36: End while
37: Output the best solution

```

4.2. Encoding and decoding**4.2.1. Priority-driven three-layer segmented encoding**

When solving DRFJSP, researchers designed the three-layer encoding scheme, including the operation sequence (OS) layer, the machine selection (MS) layer and the worker selection (WS) layer (Lou et al. 2022; Zhang, Wang, and Xu 2017; H. Zhu et al. 2020). Among them, the OS layer uses the same number to represent operations belonging to the same job, and the number of times a certain number appears indicates the operation serial number of the job corresponding to that number. Such as [3, 1, 2, 2, 1, 3] represents the operation

sequence $\{O_{31}, O_{11}, O_{21}, O_{22}, O_{12}, O_{32}\}$. Then, the MS layer arranges each operation in ascending order of the jobs, with each gene value representing the corresponding machine from the optional machine set for that operation in sequence. Taking O_{11} in Table 2 as an example, suppose its MS layer encoding value is 2, which indicates that it corresponds to the second machine (M_3) in its optional machine set $\{M_1, M_3\}$. The encoding logic of the WS layer is consistent with that of the MS layer, and each gene represents the worker further assigned to the machine selected for the operation. However, since DRFJSP-OJP considers offline operations and job priorities, the traditional three-layer encoding scheme

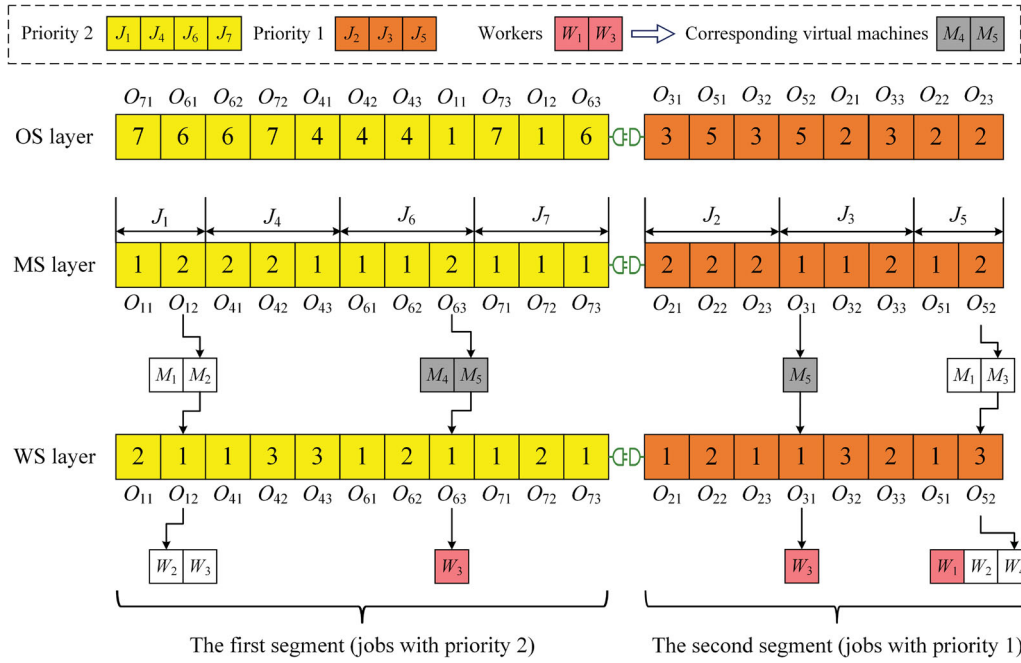


Figure 3. An example of the proposed encoding chromosome.

is no longer valid (e.g. Gnanavelbabu, Caldeira, and Vaidyanathan 2021; Han and Gong 2025; He, Tang, and Luan 2022; Usman, Lu, and Gao 2024, etc.). The main reasons are: (1) Offline operations do not need to assign machines, while the previous encoding scheme must allocate both types of resources simultaneously. (2) Existing encoding schemes typically treat all jobs as undifferentiated tasks, which cannot effectively deal with the special constraints arising from job priorities.

To address these two bottleneck issues, we propose the priority-driven three-layer segmented encoding scheme, which includes two targeted improvements based on the previous three-layer encoding method. Firstly, for the dual-mode workers that handle offline operations, configure a virtual machine one-to-one for them. For example, for the workers W_1 and W_3 in Table 2, the virtual machines M_4 (with only W_1 as the controllable worker) and M_5 (with only W_3 as the controllable worker) are added in sequence on the basis of the original machines. This approach accommodates the requirement of the three-layer encoding scheme to allocate both machine and worker resources simultaneously. For an offline operation, a virtual machine is first assigned. Since the virtual machine is bound one-to-one with the worker, this also determines the allocation of the worker at the same time. As shown in Figure 3, for O_{63} , the virtual machine assigned to it is M_5 , so it is processed independently by W_3 .

Next, for job priorities, we implement a segmented encoding for each layer of the chromosome. Specifically, jobs that belong to the same priority are encoded as a

segment of the chromosome. Thus, the number of priorities is equal to the number of segments in the chromosome. For instance, in Figure 3, the OS, MS and WS layers corresponding to the jobs $\{J_1, J_4, J_6, J_7\}$ belonging to the priority 2 are regarded as the first segment, and the jobs $\{J_2, J_3, J_5\}$ belonging to the priority 1 are regarded as the second segment. The advantage of the segmented encoding is that it facilitates the algorithm to identify jobs belonging to different priorities, allowing for higher-priority tasks to be given preference during decoding and optimisation. Therefore, through the above two key improvements, the proposed encoding scheme effectively solves the problem of chromosome expression caused by offline operations and job priorities.

4.2.2. Priority-based multi-segment active decoding

Since we have implemented segmented encoding for jobs belonging to different priorities, this makes the previous semi-active decoding method inefficient (Dauzère-Pérès et al. 2024; Shi et al. 2023; Zhang, Wang, and Xu 2017). This is because high-priority jobs can create idle time slots on machines during the semi-active decoding, which could accommodate subsequent low-priority jobs. However, the semi-active decoding method does not allow for operations to insert forward. To overcome this drawback, we propose the priority-based multi-segment active decoding scheme, which effectively improves the quality of the scheduling solution. The specific steps are as follows.

Step 1: Extract the gene segments corresponding to the jobs belong to the current highest priority in the chromosomes of the OS, MS and WS layers respectively: sub_OS , sub_MS and sub_WS , and remove them from the original chromosome.

Step 2: Extract all genes from the sub_OS chromosome. Sequentially analyse each gene to identify the target operation O_{ij} . Then, determine the processing machine M_k and the corresponding processing time p_{ijk} through the sub_MS vector, and determine the worker W_e to be allocated through the sub_WS vector.

Step 3: Find all idle time periods $[SM_{kl}, EM_{kl}]$ on machine M_k and $[SW_{el}, EW_{el}]$ for worker W_e . Here, SM , SW represent the start time of the idle time periods, while EM , EW represent the end time. On this basis, determine all the common idle time periods $[SC_l, EC_l]$ of M_k and W_e . Obviously, $SC_l = \max\{SM_{kl}, SW_{el}\}$, and $EC_l = \min\{EM_{kl}, EW_{el}\}$.

Step 4: Determine the earliest start time ES_{ij} for operation O_{ij} according to Eq. (11), where $C_{i,j-1}$ represents the end time of the previous operation. If $j = 1$, then $C_{i,j-1} = r_i$, indicating that the earliest start time for the first operation is the release time of job J_i .

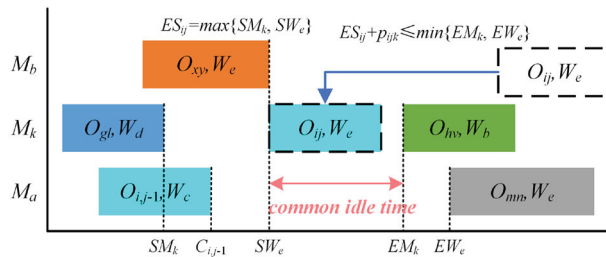
$$ES_{ij} = \max\{C_{i,j-1}, SC_l\} \quad (11)$$

Step 5: Judge whether there exists a common idle time period where O_{ij} can be inserted according to Eq. (12). If so, insert O_{ij} into the specified time period for processing, as shown in Figure 4(a), and calculate the end time $C_{ij} = ES_{ij} + p_{ijk}$.

$$ES_{ij} + p_{ijk} \leq EC_l \quad (12)$$

Step 6: If there is no common idle time period that meets the insertion condition, then schedule at t_{ij} as defined in Eq. (13), where LM_k represents the end time of the last operation of M_k , and LW_e represents the end time of the last operation of W_e . Figure 4(b) shows the scheduling of O_{ij} in the second case.

$$t_{ij} = \max\{LM_k, LW_e\} \quad (13)$$



(a) There is a common idle period that O_{ij} can be inserted

Step 7: Check whether all the genes in the sub_OS vector have been decoded. If not, return to Step 2; Otherwise, further check if there are any segments in the OS layer chromosome. If so, return to Step 1; otherwise, the decoding is completed.

4.3. Hybrid heuristic initialisation

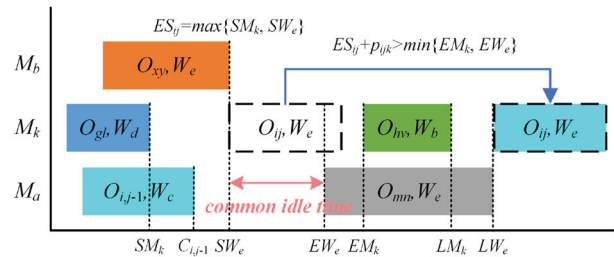
Unlike random initialisation, the heuristic approach that considers the specific characteristics of the problem can lead to fewer algorithmic iterations. Accordingly, we introduce three heuristic strategies to generate superior initial scheduling solutions.

Heuristic 1: global minimum workload heuristic considering resource continuity.

Randomly generate an OS layer chromosome and create empty chromosomes for the MS and WS layers of the same length. Then, for each operation O_{ij} in the OS layer, sequentially select the machine and worker with the minimum cumulative workload, and record their encoded values in the corresponding gene points of the MS and WS layer chromosomes. If the optional machine sets for the current operation and the previous operation include same machines, select the one with the minimum cumulative workload among these machines. Similarly, if the currently selected machine and the previous machine have the same available workers, choose the worker with the least workload among them.

Heuristic 2: local minimum workload heuristic based on jobs.

According to the job sequence of each segment in the priority-driven three-layer segmented encoding scheme, sequentially select machines and workers for each operation O_{ij} . Specifically, according to the sequence of operations for each job, select the machine and worker with the minimum cumulative workload, and store the corresponding encoded values in the MS and WS chromosomes. After initialising all operations of a job, reset the workload of each machine and worker to zero until all jobs are completed.



(b) There is no common idle period that O_{ij} can be inserted

Figure 4. A priority-based multi-segment active decoding for DRFJSP-OJP. (a) There is a common idle period that O_{ij} can be inserted. (b) There is no common idle period that O_{ij} can be inserted.

Heuristic 3: shortest processing time heuristic.

The OS layer chromosome is randomly generated, and then the MS and WS vectors of the same length are constructed. Read each operation O_{ij} of the OS layer chromosome in sequence, and select the machine with the shortest processing time from the optional machine set. For the worker selection, randomly assign a worker from the available worker set of the selected machine. Finally, store the corresponding encoded values in the MS and WS layer chromosomes.

In addition, random initialisation can maintain the diversity of the initial population. Therefore, this paper uses three heuristics and the random rule to initialise the population, with proportions of 30%, 30%, 30% and 10%, respectively.

4.4. Migration operator

The essence of the migration operator is to copy the characteristics of one individual to another one. It differs fundamentally from the crossover operator in both concept and mechanism. The former simulates the migration process of species between habitats, where individuals exchange characteristics through immigration and emigration rates, emphasising the utilisation of population information and collaboration among individuals. The latter simulates genetic recombination in biological reproduction, generating offspring by exchanging gene segments between two parent individuals, emphasising random combination and exploration. Although crossover can also be based on jobs, it typically does not distinguish between priorities, which may disrupt the job order and result in lower efficiency. In contrast, migration divides the job set into different subsets based on job priorities, allowing specific subsets of operation sequences to be copied from emigrant individuals to new individuals, thus maintaining the priority segment structure. In addition, crossover randomly produces two offspring, while migration generates only one individual that merges the excellent features of two individuals, resulting in a higher probability of producing superior solutions.

Therefore, inspired by the optimisation ability of the migration strategy of BBO, in this section, we propose a discrete migration operator to assist the population in global search and improve the convergence speed of the algorithm. Specifically, each individual is regarded as a habitat, and the greater the species number within that habitat, the more suitable for species survival, resulting in a better fitness value. All individuals in the population are ranked from best to worst, and the number of species

in each habitat is calculated using Eq. (14).

$$S_i = S_{\max} - i, \quad i = 1, 2, \dots, Ps \quad (14)$$

where, $S_{\max}$ represents the maximum number of species and is set as $S_{\max} = Ps$. According to the species number of each habitat, the immigration rate and emigration rate can be calculated. The immigration rate refers to the probability that an individual adopts the traits of others, while the emigration rate refers to the probability that the individual shares its traits with others. So, if x_j copies genetic information to x_i , the former is the emigration individual and the latter is the immigration one. Different migration rate models significantly impact the optimisation performance of the algorithm. Ma (2010) demonstrated that performance of complex migration rate models outperform simple ones. Hence, we use the cosine migration rate model to calculate the immigration rate for each individual.

$$\lambda_i = \frac{I}{2} \left(\cos \frac{\pi \cdot S_i}{S_{\max}} + 1 \right) \quad (15)$$

where, I represents the maximum immigration rate. Since it is a probability, $I = 1$. The advantage of this model is twofold. Firstly, for habitats (individuals) with medium species counts, the cosine function produces a relatively flat curve. This means that a larger portion of the population has moderate and similar immigration rates, promoting extensive information exchange and helping the algorithm to explore diverse regions. Secondly, for the best and worst habitats, the curve changes more steeply. The best individuals have very low immigration rates, protecting them from being disrupted. Conversely, the worst individuals have very high immigration rates, forcing them to learn from the better individuals.

It is worth noting that we do not calculate the emigration rate for each individual. This is because the method used by BBO to randomly select individuals for emigration based on the emigration rate has drawbacks, as it may share characteristics of individuals that are worse than x_i , potentially damaging better solutions. To address this, we propose the exemplar learning strategy to determine the emigration individual x_j for x_i using Eq. (16). Since all individuals in the population are ranked from best to worst based on their fitness values, individuals ranked higher than x_i can be considered as its exemplar individuals. This ensures that the individuals sharing characteristics with x_i are always better, thereby avoiding the selection of inferior solutions.

$$x_j : j = \text{randi}(1, i) \quad (16)$$

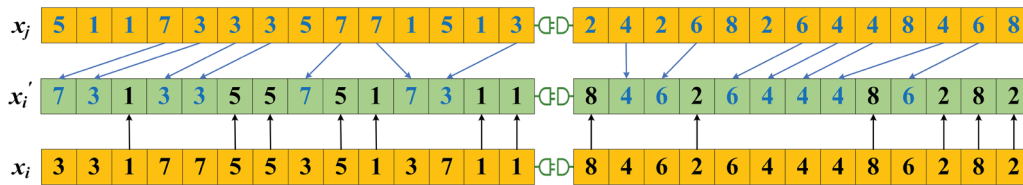


Figure 5. JBM for OS.

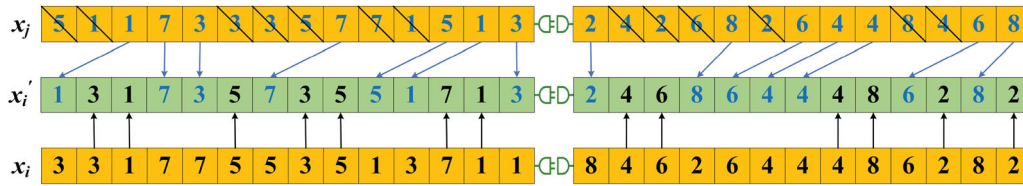


Figure 6. GPM for OS.

This exemplar learning method selects emigrants directly based on individual rankings, eliminating the need to calculate emigration rates. Due to the differences in encoding logic, different migration modes are designed for the OS, MS, and WS layer chromosomes.

For OS, randomly select one of the migration modes, either job-based migration (JBM) or generalised position migration (GPM). As shown in Figure 5, JBM performs segmented migration of the OS layer chromosomes based on the job priority, and the migration steps of each segment chromosomes are the same. For example, the job set with the highest priority is $\{J_1, J_3, J_5, J_7\}$. It is randomly divided into two non-empty subsets: $SUB_1 = \{J_1, J_5\}$, $SUB_2 = \{J_3, J_7\}$. The new individual x_i' retains the operations from x_i that belong to SUB_1 , while replicating the operations from x_j that belong to SUB_2 . Specifically, all the operations that belong to SUB_1 in the immigration individual x_i are copied to x_i' while maintaining their original positions. Then, all the operations belonging to SUB_2 in the emigration individual x_j are sequentially copied into the remaining empty positions of x_i' in their original order. For the segmentation of other priorities, the steps are the same.

For another migration model of OS, Figure 6 presents an example of GPM. Unlike JBM, which groups jobs, GPM randomly selects one sub-sequence V from the chromosome of x_i and replicates it to the same gene

points of x_i' . As shown in the figure, for the first chromosome segment, $V = [3, 1, 5, 3, 5, 7, 1]$. Then, each element of V is removed from its corresponding position in x_j , taking care to maintain the correct order. For example, if the element '7' in V is the third occurrence in x_i , then find the element '7' that appears for the third time in x_j and delete it. After removing V , the chromosome sequence of x_j becomes $[1, 7, 3, 7, 5, 1, 3]$, which is then sequentially inserted into the offspring x_i' . Similarly, for the second segment, $V = [4, 6, 4, 8, 2, 2]$, the operation sequence after x_j deletes V is $[2, 8, 6, 4, 4, 6, 8]$, and finally it is inserted into the empty position of x_i' in order. Both migration methods ensure the feasibility of new solutions, and effectively traverse the solution space of the problem.

For the MS and WS layers, the multi-point migration (MPM) mode is used simultaneously to ensure the coupling relationship of resources. Specifically, a 0-1 vector R with the same length as the chromosome is randomly generated. The gene points in x_i corresponding to '0' in R are replicated into the offspring x_i' . Then, the gene points corresponding to '1' in x_j are migrated to x_i' . As shown in Figure 7, for the first chromosome segment, $R = [0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1]$, which is still used by the first chromosome segment in the WS layer during migration. This method can traverse different resource allocation strategies, and enhance the diversity of scheduling schemes.

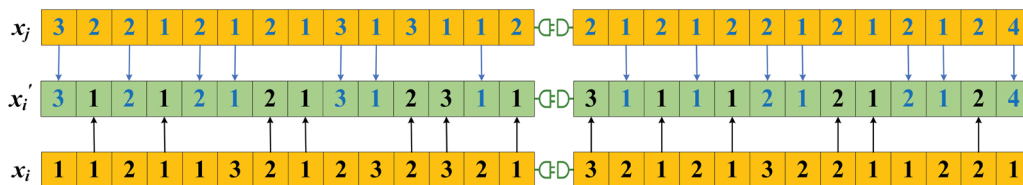


Figure 7. MPM for MS.

4.5. Mutation operator

As a complement to the migration operator, mutation provides the population with the opportunity to escape local optima, helping to prevent premature convergence. So, this paper also designs different mutation operators specifically for the OS, MS, and WS layers.

For the OS layer, the inversion mutation strategy is employed. From each chromosome segment, two gene points r_1 and r_2 are randomly selected, and all gene points between r_1 and r_2 are inversed. As shown in Figure 8, for the first chromosome segment of the OS layer, the gene sequence [3, 1, 3, 5, 7] between $r_1 = 5$ and $r_2 = 9$, is selected. After the inversion mutation, the new gene arrangement becomes [7, 5, 3, 1, 3]. Similarly, for the second chromosome segment, the gene sequence [8, 4, 6, 2] of the mutant individual is generated by reversing the original segment [2, 6, 4, 8].

For the MS layer, a gene point is randomly selected from each chromosome segment and replaced with the machine with the shortest processing time among the optional machine set for that operation. For the WS layer, one position is selected from each chromosome segment, and a worker is randomly assigned from the eligible worker set. If the operation corresponding to this gene point can only be processed by one machine or worker, then randomly select a new position.

The proposed mutation operators increase population diversity, enabling the exploration of different regions in the solution space and potentially leading to better overall optimisation results.

4.6. Problem-specific neighbourhood search

Neighborhood search is the bridge between current solutions and potentially better ones. In this section, the unique global critical-path of DRFJSP-OJP is proposed, along with multiple local critical-paths based on job priorities. Furthermore, the neighbourhood structure driven by common critical operations is designed.

4.6.1. Collaboration of multi-critical-path

Since DRFJSP-OJP introduces job release time and worker resource, the critical path of traditional FJSP is no longer applicable. So, we also take the continuity of workers in time as a factor for identifying the critical

path. Specifically, for the worker assigned to the current critical operation, if the previous operation performed by this worker is continuous in time with the current critical operation, then the previous operation can also be defined as a critical operation. In addition, both the global critical path and the local critical path are defined for a job set. For example, the local critical path corresponding to the highest priority is focussed on the set of jobs that belong to that highest priority category. In contrast, the global critical path clearly pertains to the collection of all jobs involved in the scheduling problem. This means that whether it is the global or local critical-path, only the corresponding job set needs to be determined, because the definition method of the critical path is the same. Based on this, Algorithm 2 presents the specific steps for determining the critical path.

As shown in Figure 9, this gantt chart presents an example of multiple critical-paths. The global critical path obtained from the set of all jobs is $O_{71} \rightarrow O_{72} \rightarrow O_{53} \rightarrow O_{54} \rightarrow O_{55} \rightarrow O_{34} \rightarrow O_{41} \rightarrow O_{42} \rightarrow O_{43} \rightarrow O_{44} \rightarrow O_{45} \rightarrow O_{26} \rightarrow O_{83} \rightarrow O_{15} \rightarrow O_{10,7} \rightarrow O_{86}$. In this scheduling scheme, the last completed operation O_{86} among all jobs is identified as a critical operation, and other critical operations are searched forward in sequence. If the start time of the critical operation O_{ij} is equal to the end time of its preceding operation $O_{i,j-1}$, or equal to the completion time of the previous operation on the same machine PM_{ij} , or equal to the end time of the previous task assigned to the same worker PW_{ij} , then the corresponding preceding operation is also defined as a critical operation. According to the same method, local critical-paths belonging to the job sets of different priorities can be defined. For instance, for the jobs $\{J_3, J_5, J_7\}$ belonging to the highest priority, determine the latest completed operation O_{35} as the critical operation, and then search forward in sequence to obtain the local critical-path: $O_{71} \rightarrow O_{72} \rightarrow O_{53} \rightarrow O_{54} \rightarrow O_{55} \rightarrow O_{56} \rightarrow O_{75} \rightarrow O_{76} \rightarrow O_{35}$. Similarly, the local critical-path corresponding to jobs with the second priority is $O_{41} \rightarrow O_{42} \rightarrow O_{43} \rightarrow O_{44} \rightarrow O_{45} \rightarrow O_{26} \rightarrow O_{27} \rightarrow O_{28}$. It can be observed that all critical operations of the local critical-path with the lowest priority will be included in the global critical-path, as they share the same maximum completion time. Finally, by comparing each local critical-path with the global critical-path, we can

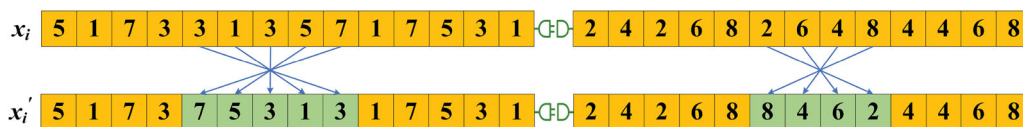


Figure 8. Inversion mutation for OS.

Algorithm 2 Pseudo code for calculating the critical-path

```

1: Determine the job set  $U$  of the critical path
2: Define  $ST(\cdot)$  and  $CT(\cdot)$  to be the start time and end time of an operation
3: Define operation  $O_{ij}$  to be processed on machine  $M_{ij}$ , and  $M_{ij}$  to be operated by worker  $W_{ij}$ 
4: Define the previous operations of  $O_{ij}$ ,  $M_{ij}$ , and  $W_{ij}$  as  $PO_{ij}$ ,  $PM_{ij}$ , and  $PW_{ij}$ , respectively
5: Define the critical operation set  $C_{op} = \emptyset$ 
6: Define  $O_{ij}$  as a critical operation  $CO_{ij}$  if  $CT(O_{ij})$  is equal to the maximum completion time in the job set  $U$ 
7: If  $CO_{ij}$  requires machine processing
8:   Define  $ET$  as the earliest start time of machine  $M_{ij}$ 
9: Else
10:   Define  $ET$  as the earliest start time of worker  $W_{ij}$ 
11: End If
12:  $C_{op} = C_{op} \cup CO_{ij}$ 
13: While ( $ST(CO_{ij}) \neq ET$ ) do
14:   If  $j \geq 2$ 
15:     If  $CO_{ij}$  requires machine processing
16:       Define  $O_{i'j'}$  as a critical operation  $CO_{i'j'}$  with  $CT = \max\{CT(PO_{ij}), CT(PM_{ij}), CT(PW_{ij})\}$ 
17:     Else
18:       Define  $O_{i'j'}$  as a critical operation  $CO_{i'j'}$  with  $CT = \max\{CT(PO_{ij}), CT(PW_{ij})\}$ 
19:     End If
20:   Else
21:     If  $CO_{ij}$  requires machine processing
22:       Define  $O_{i'j'}$  as a critical operation  $CO_{i'j'}$  with  $CT = \max\{CT(PM_{ij}), CT(PW_{ij})\}$ 
23:     Else
24:       Define  $O_{i'j'}$  as a critical operation  $CO_{i'j'}$  with  $CT = CT(PW_{ij})$ 
25:     End If
26:   End If
27:    $O_{i'j'} \rightarrow O_{ij}$ ,  $M_{i'j'} \rightarrow M_{ij}$ ,  $W_{i'j'} \rightarrow W_{ij}$ 
28:    $C_{op} = C_{op} \cup CO_{ij}$ 
29: End While
30: Output the critical operation set  $C_{op}$  and critical path

```

extract the critical operations that are common to both, and the common critical operation set is obtained as $\{O_{71}, O_{72}, O_{53}, O_{54}, O_{55}, O_{41}, O_{42}, O_{43}, O_{44}, O_{45}, O_{26}, O_{83}, O_{15}, O_{10,7}, O_{86}\}$. The extraction process of common critical operations is shown in Algorithm 3.

4.6.2. Neighborhood structure driven by common critical operations

Based on the set of the above common critical operations, an efficient neighbourhood structure that satisfies the resource coupling constraints is designed. This neighbourhood structure aims to reduce the makespan by strategically repositioning critical operations, thereby optimising resource utilisation and enhancing overall scheduling efficiency.

For the critical operation O_{ij} , let PO_{ij} denote its preceding operation, and NO_{ij} denote its next operation. Suppose the operation O_{ij} is processed by the

machine M_k , we define PM_{ij}^k as the previous operation on machine M_k , and NM_{ij}^k as the next task on the same machine. Then, suppose the operation O_{ij} is performed by worker W_e , then PW_{ij}^e represents the previous task of the worker W_e , and NW_{ij}^e represents the succeeding operation of the worker. Clearly, the earliest start time ST^E of this critical operation O_{ij} equals the maximum value of the latest completion time CT^L of PO_{ij} , PM_{ij}^k , and PW_{ij}^e . Thus, we have $ST^E(O_{ij}) = \max\{CT^L(PO_{ij}), CT^L(PM_{ij}^k), CT^L(PW_{ij}^e)\}$. Conversely, the latest completion time of the critical operation O_{ij} is equal to the minimum value of the earliest start time of NO_{ij} , NM_{ij}^k , and NW_{ij}^e , which can be expressed as $CT^L(O_{ij}) = \min\{ST^E(NO_{ij}), ST^E(NM_{ij}^k), ST^E(NW_{ij}^e)\}$.

Based on the generalised definition above, we can begin moving critical operations to conduct the neighbourhood search. For the critical operation O_{ij} , it can only be moved if there is an idle time period for insertion. Therefore, we search for these idle time periods

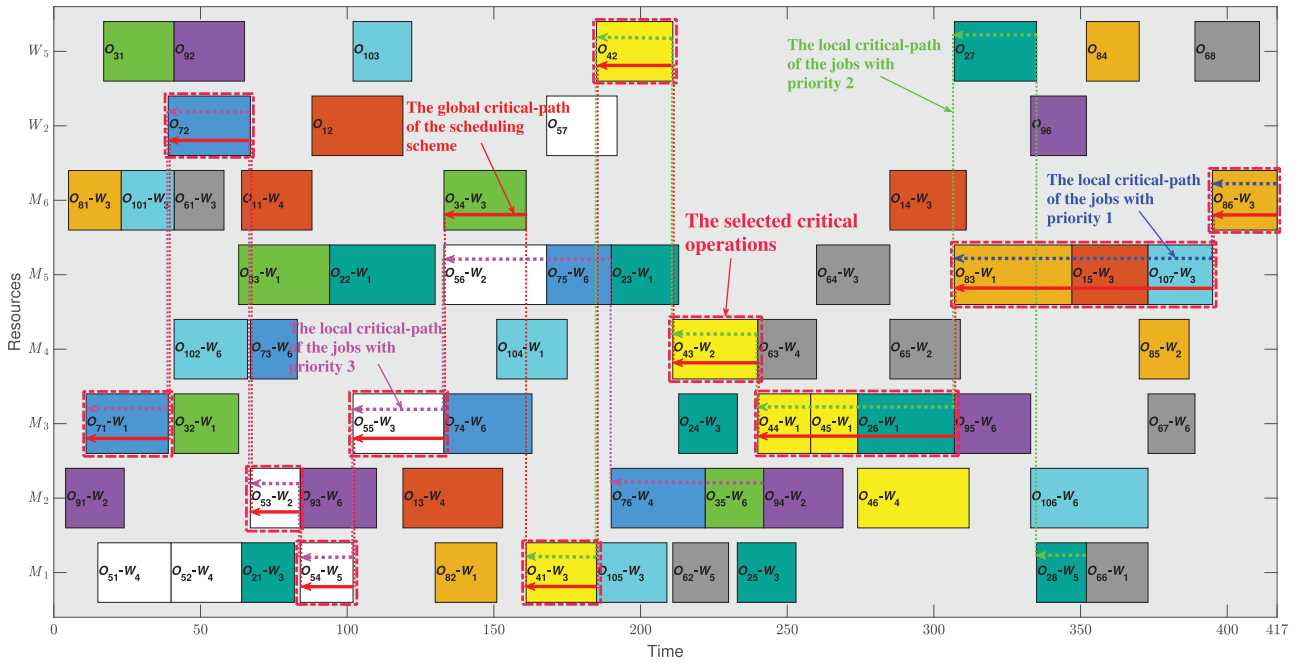


Figure 9. A diagram of a critical path.

by exploring the optional machines and corresponding resources for O_{ij} . However, since some critical operations may not require machine processing and only need the worker resource, the conditions for their insertion will differ.

Firstly, if the critical operation O_{ij} requires the machine resource, the idle time of both machines and workers needs to be considered simultaneously. The earliest start time ST^E of the critical operation O_{ij} can be determined by the following formula.

$$ST^E(O_{ij}) = \max \left\{ CT^L(PO_{ij}), CT^L(PM_{ij}^k), CT^L(PW_{ij}^e) \right\} \quad (17)$$

Then, if $ST^E(O_{ij})$ meets the following insertion condition, O_{ij} can be moved to that idle time period.

$$ST^E(O_{ij}) + p_{ijk} \leq \min \left\{ ST^E(NO_{ij}), ST^E(NM_{ij}^k), ST^E(NW_{ij}^e) \right\} \quad (18)$$

If the critical operation O_{ij} does not require machine processing, then only the availability of the worker W_e needs to be considered. In this case, the earliest start time of O_{ij} can be determined by Eq. (19).

$$ST^E(O_{ij}) = \max \left\{ CT^L(PO_{ij}), CT^L(PW_{ij}^e) \right\} \quad (19)$$

Furthermore, only when $ST^E(O_{ij})$ satisfies the constraint of Eq. (20) can operation O_{ij} be inserted into the corresponding idle time period.

$$ST^E(O_{ij}) + p_{ijk} \leq \min \left\{ ST^E(NO_{ij}), ST^E(NW_{ij}^e) \right\} \quad (20)$$

The neighbourhood structure driven by common critical operations takes into account the dual-resource flexibility, and conducts precise perturbations for different types of critical operations. Algorithm 3 presents its pseudo code. This method helps the algorithm identify the better scheduling scheme by systematically searching for neighbourhood solutions, thereby improving the overall optimisation performance.

4.7. Complexity analysis

The time complexity of MCPEA mainly depends on the population size Ps , the number of iterations G , and the total number of operations N , where the total length of chromosomes is $3N$. The initialisation generates the population using the hybrid heuristic method, with a complexity of $O(Ps \cdot 3N) \approx O(Ps \cdot N)$. Similarly, the fitness evaluation involves decoding and calculating the objective function, with a complexity of $O(Ps \cdot N)$ per generation. Both the migration and mutation operators have a complexity of $O(Ps \cdot N)$. The neighbourhood search includes the critical-path calculation with $O(N)$ and the extraction of common critical operations with $O(N)$,

Algorithm 3 Pseudo code for common critical operations driven neighbourhood search

-
- 1: Determine the current scheduling solution X
 - 2: Define X' to be a new neighboring solution
 - 3: Initialize the common critical operation set $C_{common} \leftarrow \emptyset$
 - 4: Calculate the global critical-path CP_{global} for all jobs using Algorithm 2
 - 5: Extract the global critical operation set C_{global} from CP_{global}
 - 6: For (each job priority level p) do
 - 7: Determine the job set U_p belonging to priority p
 - 8: Calculate the local critical-path CP_{local}^p for U_p using Algorithm 2
 - 9: Extract the local critical operation set C_{local}^p from CP_{local}^p
 - 10: $C_{common} \leftarrow C_{common} \cup (C_{global} \cap C_{local}^p)$
 - 11: End for
 - 12: Remove duplicates from C_{common}
 - 13: Randomly select a critical operation O_{ij} from C_{common}
 - 14: If (O_{ij} requires machine processing) do
 - 15: Identify available machine M_k and worker W_e for O_{ij}
 - 16: Calculate $ST^E(O_{ij})$ using Equation (17)
 - 17: If ($ST^E(O_{ij})$ satisfies Equation (18)) do
 - 18: Move O_{ij} to the idle time period on M_k and W_e
 - 19: Update the schedule to generate X'
 - 20: End if
 - 21: Else % Only requires worker resource
 - 22: Identify available worker W_e for O_{ij}
 - 23: Calculate $ST^E(O_{ij})$ using Equation (19)
 - 24: If ($ST^E(O_{ij})$ satisfies Equation (20)) do
 - 25: Move O_{ij} to the idle time period on W_e
 - 26: Update the schedule to generate X'
 - 27: End if
 - 28: End if
 - 29: Output the new solution X'
-

resulting in an overall complexity of $O(Ps \cdot N)$. The insertion check in the critical operation movement takes $O(1)$, which is neglected.

Therefore, the overall time complexity of MCPEA can be approximated as $O(G \cdot Ps \cdot N)$.

5. Experiments and discussions

In this section, to fully test the performance of MCPEA in solving the DRFJSP-OJP, systematic numerical experiments are conducted. 20 benchmark problems are

used as the test suite. To ensure fairness, the same maximum running time is used as the termination condition for all algorithms. All experiments are conducted on the system equipped with an Intel(R) Core(TM) i7-14700KF processor running at 3.40 GHz and 16.0 GB RAM, and the experimental platform includes MATLAB R2022a and IBM ILOG CPLEX 20.1.0 solver.

In addition, to assess the performance of encoding and decoding schemes when handling job priority constraints, we designed weighted completion time difference with release time compensation as a evaluation metric of priority satisfaction degree (PSD). As shown in Eq. (21), subtracting the release time avoids a larger completion time caused by delayed task releases, and multiplying by the priority weight makes the delay penalty for high-priority jobs greater. So, PSD reflects whether high-priority jobs are completed as quickly as possible after being released, thus giving higher weight to high-priority jobs. A smaller value of PSD indicates a better adherence to job priority constraints in the scheduling solution, ensuring that high-priority jobs are processed preferentially.

$$PSD = \frac{\sum_{i=1}^n (p_i \cdot (C_i - r_i))}{\sum_{i=1}^n p_i} \quad (21)$$

5.1. Benchmarks construction

Based on the actual production data of a certain enterprise, this paper generates a test suite named ZQL, which includes 20 benchmark instances (ZQL01-ZQL20). This test suite includes problems of different scales, with the number of jobs increasing from six to 30, machines increasing from five to 12, job priorities increasing from two to five, workers increasing from four to 10. Table 4 presents the detailed information about the test suite. In this table, n represents the number of jobs, m represents the number of machines, and w represents the total number of workers. Furthermore, $[min, max]$ represents the minimum and maximum operation numbers in the job set, sum_op represents the total number of operations in this instance, avg_op represents the average value of operations for each job, pro_off represents the proportion of offline operations in the problem. Furthermore, 'CPU (s)' represents the running time (in seconds) of each algorithm for this instance, that is, the termination criterion of each algorithm. Its calculation formula is 'ceil($3 \cdot sum_op/5$) $\cdot 5$ '. The termination time for each instance is set according to its total number of operations.

Taking ZQL03 as an example, it contains 10 jobs, five machines, and five workers, with a total of 56 operations. The jobs contain a minimum of four operations and a maximum of eight operations, with an average of 5.60

Table 4. Descriptions of the benchmark problems (ZQL01–ZQL20).

No.	n	m	w	[min, max]	sum_op	avg_op	pro_off	CPU (s)
ZQL01	6	5	4	[4, 6]	29	4.83	13.79%	45
ZQL02	8	5	4	[5, 6]	42	5.25	11.90%	65
ZQL03	10	5	5	[4, 8]	56	5.60	26.79%	85
ZQL04	10	6	6	[5, 8]	64	6.40	26.56%	100
ZQL05	11	7	6	[4, 7]	62	5.64	29.03%	95
ZQL06	12	6	5	[5, 6]	71	5.92	23.94%	110
ZQL07	14	7	5	[6, 7]	90	6.43	26.67%	135
ZQL08	16	8	6	[5, 9]	103	6.44	24.27%	155
ZQL09	16	8	7	[4, 8]	105	6.56	28.57%	160
ZQL10	17	7	4	[8, 8]	136	8.00	8.09%	205
ZQL11	18	8	6	[7, 9]	145	8.06	23.45%	220
ZQL12	20	8	7	[6, 9]	148	7.40	21.62%	225
ZQL13	21	9	8	[5, 10]	148	7.05	22.97%	225
ZQL14	22	10	8	[6, 8]	154	7.00	23.38%	235
ZQL15	23	10	8	[6, 7]	150	6.52	37.33%	225
ZQL16	24	10	7	[5, 9]	166	6.92	18.07%	250
ZQL17	26	10	8	[7, 8]	194	7.46	26.29%	295
ZQL18	28	11	7	[4, 9]	172	6.14	16.28%	260
ZQL19	30	11	9	[7, 7]	210	7.00	20.95%	315
ZQL20	30	12	10	[6, 10]	234	7.80	22.22%	355

operations per job. Additionally, 26.79% of all operations are offline operations. When calculating this instance using algorithms, the running time is 85 seconds. The detailed data of the ZQL benchmark suite and the corresponding MATLAB code have been made available at <https://zenodo.org/records/17226700>.

5.2. Parameters setting

The proposed MCPEA involves three parameters that need to be initialised: population size (Ps), mutation probability (Mp), and local search probability (Lp). To allow non-experts to execute the algorithm by setting parameters without needing to understand the specific details of the algorithm, this paper uses the Taguchi experiment (Z. Zhang, Li et al. 2025) to determine the parameter combinations. Firstly, we provide four candidate values for each parameter: $Ps = [50, 100, 150, 200]$, $Mp = [0.05, 0.10, 0.15, 0.20]$, and $Lp = [0.1, 0.2, 0.3, 0.4]$. Then, according to the number of parameters and levels, the orthogonal table L_{16} is determined, as shown in Table 5, resulting in 16 combinations of the three parameters. Each set of parameters is applied to the MCPEA to solve the 20 instances of the ZQL test suite. The CPU running time from Table 4 serves as the termination criterion, and the average value of the objective function obtained by the algorithm for each parameter set is recorded as the response value.

Finally, Figure 10 shows the average response values of the three parameters at the same level. It is not difficult to see that the optimal levels for each parameter are L_2 , L_3 , and L_3 , respectively. Therefore, the recommended optimal parameter combination for MCPEA is $[Ps, Mp, Lp] = [100, 0.15, 0.3]$.

Table 5. The orthogonal table of the Taguchi experiment.

No.	Ps	Mp	Lp	Response value
1	50	0.05	0.1	851.8
2	50	0.10	0.2	858.6
3	50	0.15	0.3	851.7
4	50	0.20	0.4	857.4
5	100	0.05	0.2	854.5
6	100	0.10	0.1	855.4
7	100	0.15	0.4	852.5
8	100	0.20	0.3	846.3
9	150	0.05	0.3	854.6
10	150	0.10	0.4	858.2
11	150	0.15	0.1	853.8
12	150	0.20	0.2	857.0
13	200	0.05	0.4	855.7
14	200	0.10	0.3	856.5
15	200	0.15	0.2	851.4
16	200	0.20	0.1	854.1

5.3. Ablation experiment of the algorithm

This subsection performs an ablation experiment to validate the effectiveness of the key components in MCPEA. We design three algorithm variants for comparison. MCPEA-V1 utilises random initialisation instead of the hybrid heuristic method. MCPEA-V2 replaces the migration operator with the traditional crossover operator. MCPEA-V3 disables the problem-specific neighbourhood search to assess its contribution to solution improvement. All variants maintain identical parameter settings and termination criteria as the complete MCPEA. All algorithms are independently run 20 times on 20 benchmark instances to ensure the scientific nature of the experiments. The best, mean value of the makespan and PSD are recorded as the experimental results, as given in Table 6, where the bold data represents the best results among all variants.

According to the experimental results, the complete MCPEA achieves the best average makespan on 85 percent instances and obtains the best average PSD value on 90 percent problems, outperforming the three variants comprehensively. Specifically, MCPEA-V1 with random initialisation shows a major decline in average solution quality on 75 percent of the problems, demonstrating the effectiveness of the hybrid heuristic initialisation in improving the convergence starting point. MCPEA-V2 using the crossover operator performs moderately well on instances like ZQL19, but its degree of priority satisfaction is inferior to that of the migration operator on most problems, verifying the advantages of the migration strategy based on exemplar learning. MCPEA-V3 without neighbourhood search experiences the most significant performance decline, performing obviously worse on complex problems, which highlights the importance of neighbourhood search based on multiple critical-paths in developing better solutions. It is evident that all three

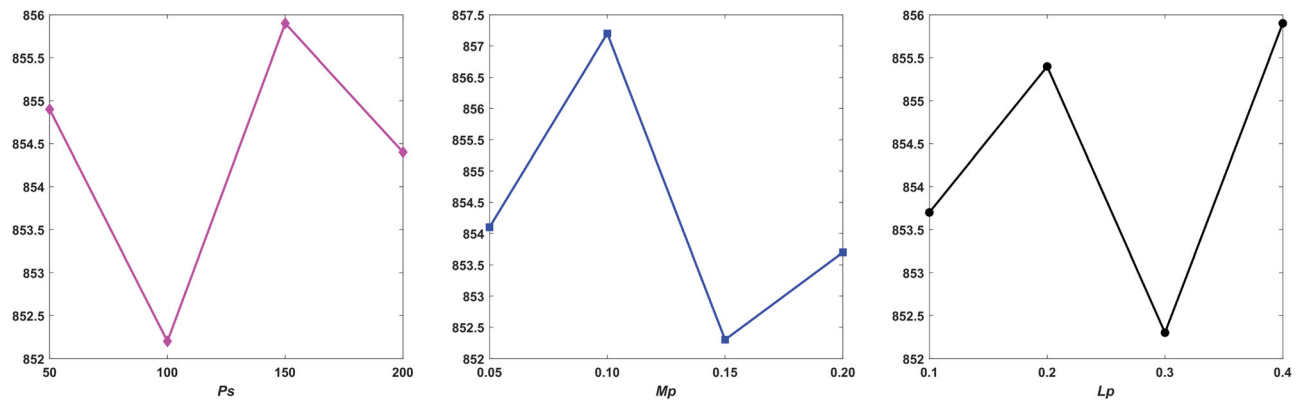


Figure 10. The mean response values at the same level of three parameters.

Table 6. Results of MCPEA-V1, MCPEA-V2, MCPEA-V3 and MCPEA.

No.	Indexs	MCPEA-V1		MCPEA-V2		MCPEA-V3		MCPEA	
		Best	Mean	Best	Mean	Best	Mean	Best	Mean
ZQL01	Makespan	243	254.5	245	251.9	244	255.4	239	249.5
	PSD	201.6	211.1	194.7	209.9	199.0	208.5	197.7	205.3
ZQL02	Makespan	332	342.4	327	340.4	333	347.6	324	340
	PSD	216.1	239.4	213.1	229.1	222.2	234.7	212.3	226.7
ZQL03	Makespan	306	324.6	315	320.6	312	323.2	305	316.2
	PSD	188.2	198.7	189.4	199.4	183.3	201.5	178.8	192.5
ZQL04	Makespan	359	365.9	357	369.3	374	387.4	347	368.5
	PSD	215.7	224.1	222.7	226.2	229.2	239.3	198.1	222.7
ZQL05	Makespan	338	346.1	337	343.7	343	354.4	335	346
	PSD	229.5	251.0	234.1	239.4	238.5	254.1	229.3	238.9
ZQL06	Makespan	480	489.1	474	484.2	502	511	477	491.3
	PSD	315.4	327.8	298.2	332.1	327.4	341.2	304.7	323.7
ZQL07	Makespan	509	521.6	516	524.2	541	549.8	506	518.9
	PSD	302.7	316.4	303.1	322.0	327.0	339.5	297.9	312.9
ZQL08	Makespan	589	596.6	583	592.9	603	616	566	587.3
	PSD	328.8	346.4	331.5	356.8	348.1	368.9	323.6	341.7
ZQL09	Makespan	575	584.7	572	591.6	621	631.5	564	581.2
	PSD	264.9	287.4	270.3	287.4	287.5	309.1	268.2	282.9
ZQL10	Makespan	1062	1083.1	1058	1075.9	1102	1121.4	1048	1078.7
	PSD	550.7	584.2	536.0	569.3	578.1	594.9	529.0	563.0
ZQL11	Makespan	833	862.7	849	862.9	891	903.2	826	854.1
	PSD	486.5	507.3	469.2	499.3	479.9	515.6	456.8	481.3
ZQL12	Makespan	851	892.4	890	910.3	922	953.8	838	887.4
	PSD	394.8	420.2	401.7	426.8	426.5	452.2	400.2	417.6
ZQL13	Makespan	818	838.6	813	834.8	861	875.2	783	825
	PSD	459.8	483.9	461.1	485.5	476.3	505.4	462.6	472.4
ZQL14	Makespan	794	836.4	819	836.7	866	880.8	792	827.2
	PSD	450.9	484.2	444.5	474.0	476.4	504.6	439.1	462.8
ZQL15	Makespan	709	736.6	726	741.8	786	794.3	695	724.8
	PSD	361.9	374.3	353.7	371.7	383.1	399.0	336.5	359.9
ZQL16	Makespan	934	955.9	950	963.8	988	1003.3	908	954.5
	PSD	518.4	544.4	526.3	545.2	533.4	568.9	517.6	532.7
ZQL17	Makespan	824	855.4	839	865.2	887	899	811	840.3
	PSD	378.7	397.8	381.5	394.6	382.1	405.5	375.8	383.0
ZQL18	Makespan	993	1012.4	1002	1020.2	1028	1044.1	986	1008.4
	PSD	554.1	570.3	526.0	554.8	549.0	580.9	522.3	554.7
ZQL19	Makespan	891	899.7	881	896.2	910	929.8	886	894.5
	PSD	518.1	536.1	511.4	530.7	513.4	536.8	498.4	514.3
ZQL20	Makespan	997	1015.6	1002	1014.3	1028	1036.3	978	1005.7
	PSD	491.3	508.7	483.5	511.8	497.9	524.2	485.4	500.6

strategies enhance the algorithm performance to different extents, with the neighbourhood structure playing the most critical role. As a result, the experimental results fully prove the necessity and collaborative effectiveness of the three components. Three strategies complement each other to jointly enhance the algorithm performance.

5.4. Comparison between the model and the algorithm

To verify the correctness of the proposed MILP model, we employ the CPLEX solver to solve the 20 instances, setting the maximum computation time for each case to 600 s. At the same time, the MCPEA is applied to

Table 7. Results obtained by MILP model and MCPEA.

No.	MILP		MCPEA		No.	MILP		MCPEA	
	Makespan	PSD	Makespan	PSD		Makespan	PSD	Makespan	PSD
ZQL01	317	222.8	239	197.7	ZQL11	NaN	NaN	826	456.8
ZQL02	194	168.3	324	212.3	ZQL12	NaN	NaN	838	400.2
ZQL03	217	165.1	305	178.8	ZQL13	NaN	NaN	783	462.6
ZQL04	324	253.5	347	198.1	ZQL14	NaN	NaN	792	439.1
ZQL05	288	249.2	335	229.3	ZQL15	NaN	NaN	695	336.5
ZQL06	348	300.4	477	304.7	ZQL16	NaN	NaN	908	517.6
ZQL07	522	483.3	506	297.9	ZQL17	NaN	NaN	811	375.8
ZQL08	672	378.0	566	323.6	ZQL18	NaN	NaN	986	522.3
ZQL09	NaN	NaN	564	268.2	ZQL19	NaN	NaN	886	498.4
ZQL10	NaN	NaN	1048	529.0	ZQL20	NaN	NaN	978	485.4

address these problems, with the termination criterion for the algorithm being the preset running time for each instance. Although the MILP model incorporates a penalty term in its objective function to handle job priority constraints, for an intuitive evaluation of solution efficiency, we record the makespan and PSD of the optimal scheduling solution for comparison, as shown in Table 7. Here, ‘NaN’ stands for ‘Not a Number’, indicating that the MILP model cannot find any feasible solution for the corresponding instance within the 600-s time limit.

From Table 7, it is clear that for small-scale instances, the proposed MILP model can find feasible solutions within the 600 s, especially achieving better completion time than MCPEA on ZQL04, ZQL05, and ZQL06. This verifies the correctness of the constructed MILP model and indicates that the model can accurately describe and solve the DRFJSP-OJP. However, as the problem size increases, this model fails to find feasible solutions for any larger instances within 600 s. In contrast, the MCPEA demonstrates superior solving efficiency, generating high-quality feasible scheduling schemes for instances of varying sizes within a reasonable time. Therefore, experimental results confirm the correctness of the MILP model and its ability to successfully solve some small-scale problems. But it struggles to obtain feasible solutions for larger instances even with long time.

5.5. Comparison between the algorithm and other methods

To verify the superiority of MCPEA, comparative experiments are conducted with eGA (Zhang, Gao, and Shi 2011), HDPSO (Zhang, Wang, and Xu 2017), HSDE (Li, Wang, and Peng 2022), DCGA (Han et al. 2024), and KMA (Deng et al. 2025). The selection of these algorithms is not arbitrary but is based on the high relevance of the problems addressed in their original literature to the core challenges of this paper, ensuring fairness and persuasiveness in the comparisons. The specific rationale is as follows: (1) eGA is a classic and

Table 8. Parameters setting for all compared algorithms.

Algorithm	Parameters setting	Termination condition
eGA	$P_s = 100$; $r_{GS} = 0.6$; $r_{LS} = 0.3$; $r_{RS} = 0.1$; $P_c = 0.7$; $M_p = 0.01$	CPU (s)
HDPSO	$P_s = 100$; $T_0 = 3$; $T_{end} = 0.01$; $B = 0.9$; $p_{l1} = 0.6$; $p_{l2} = 0.8$; $pf_{max} = 0.9$; $pf_{min} = 0.2$	CPU (s)
HSDE	$P_s = 200$; $F = 0.7$; $CR = 0.2$	CPU (s)
DCGA	$P_s = 500$; $P_c = 0.9$; $M_p = 0.15$	CPU (s)
KMA	$P_s = 200$; $P_c = 0.9$; $M_p = 0.6$; $\alpha = 0.92$	CPU (s)
MCPEA	$P_s = 100$; $M_p = 0.15$; $L_p = 0.3$	CPU (s)

efficient algorithm for solving standard FJSP. It is chosen as a reference benchmark to demonstrate that specialised improvements for the complex constraints in this paper must significantly outperform established general algorithms. (2) HDPSO is an advanced evolutionary algorithm specifically designed for DRFJSP. It directly tests whether MCPEA performs better than existing advanced algorithms in the same field when addressing the core challenge of coordinating scheduling between machines and workers. (3) HSDE is one of the few studies that explicitly considers job priority constraints in DRFJSP. Therefore, it serves as the most relevant comparison method for evaluating MCPEA’s performance in meeting priority constraints. (4) DCGA and KMA represent the latest algorithmic frameworks for solving complex resource-constrained problems. They can verify whether the overall framework design of MCPEA has advantages over the current efficient algorithms when facing complex extended FJSP. Then, the parameters for the five algorithms are set according to their original reference papers, as given in Table 8.

Similarly, all algorithms are independently run 20 times on each problem. The best value and mean value of the makespan, along with their corresponding PSD, are recorded, as shown in Table 9, where the bold data represents the best results. It is not difficult to see that the proposed algorithm demonstrates outstanding overall performance in optimising makespan under the same CPU time constraint. From Table 9, it can be observed that MCPEA achieves the best makespan on 14 instances

Table 9. Results of eGA, HDPSO, HSDE, DCGA, KMA and MCPEA.

No.	Indexs	eGA		HDPSO		HSDE		DCGA		KMA		MCPEA	
		Best	Mean	Best	Mean	Best	Mean	Best	Mean	Best	Mean	Best	Mean
ZQL01	Makespan	284	298.6	269	277.5	250	265.7	246	260.0	247	256.0	239	249.5
	PSD	198.9	224.5	201.3	215.7	207.1	219.1	205.0	215.3	207.6	220.1	197.7	205.3
ZQL02	Makespan	359	366.9	347	359.4	338	366.9	335	347.6	336	341.5	324	340.0
	PSD	248.8	257.0	214.4	227.1	237.3	252.4	237.3	249.4	236.6	253.3	212.3	226.7
ZQL03	Makespan	321	342.8	321	325.3	331	350.0	313	324.7	307	318.1	305	316.2
	PSD	201.3	208.4	193.8	199.1	198.6	214.8	199.1	208.0	193.1	209.3	178.8	192.5
ZQL04	Makespan	397	413.5	387	397.1	404	412.4	368	377.6	356	370.6	347	368.5
	PSD	234.6	244.0	228.1	240.1	227.8	254.1	216.6	237.1	232.4	244.3	198.1	222.7
ZQL05	Makespan	367	371.1	360	368.7	370	379.4	336	349.6	332	340.7	335	346.0
	PSD	245.6	254.6	243.9	253.4	249.3	268.7	237.8	256.7	234.2	253.4	229.3	238.9
ZQL06	Makespan	484	496.1	505	517.6	502	528.3	489	501.4	475	486.9	477	491.3
	PSD	340.2	346.8	314.2	327.3	333.0	351.9	319.7	343.7	304.2	337.5	304.7	323.7
ZQL07	Makespan	550	563.0	533	539.6	552	570.3	519	532.7	504	520.2	506	518.9
	PSD	309.6	324.5	314.8	326.2	324.6	353.7	316.6	336.8	312.2	333.9	297.9	312.9
ZQL08	Makespan	592	622.7	611	620.7	632	645.4	586	612.7	570	593.7	566	587.3
	PSD	339.9	358.7	351.5	363.6	346.4	372.9	349.3	369.8	332.0	348.1	323.6	341.7
ZQL09	Makespan	618	633.6	594	607.4	632	651.8	589	611.1	582	592.7	564	581.2
	PSD	293.3	297.2	277.5	295.2	308.3	320.8	288.9	307.1	292.7	309.1	268.2	282.9
ZQL10	Makespan	1098	1118.9	1080	1094.8	1114	1156.0	1076	1104.6	1070	1098.0	1048	1078.7
	PSD	580.8	602.8	521.5	532.0	614.4	632.9	584.3	612.4	568.1	587.5	529.0	563.0
ZQL11	Makespan	854	883.0	855	871.9	929	941.7	874	884.4	847	875.6	826	854.1
	PSD	490.3	512.7	468.9	482.6	505.6	538.2	517.6	545.3	479.1	516.8	456.8	481.3
ZQL12	Makespan	925	944.5	902	913.9	973	991.2	879	923.7	896	920.9	838	887.4
	PSD	425.3	444.1	412.1	430.2	441.3	469.2	403.4	444.8	442.8	457.6	400.2	417.6
ZQL13	Makespan	827	867.0	832	849.6	884	904.7	836	851.4	813	836.7	783	825.0
	PSD	476.0	502.1	469.9	482.1	499.2	537.5	500.5	527.7	473.9	503.2	462.6	472.4
ZQL14	Makespan	854	865.6	842	851.9	907	917.3	832	853.8	835	847.3	792	827.2
	PSD	461.3	478.2	444.6	464.8	502.7	521.9	472.8	497.6	460.2	499.5	439.1	462.8
ZQL15	Makespan	769	788.3	738	757.1	821	838.3	762	774.7	719	741.0	695	724.8
	PSD	382.1	390.3	357.7	364.5	394.9	409.4	379.1	399.4	369.7	402.1	336.5	359.9
ZQL16	Makespan	950	973.2	953	969.6	1015	1039.8	960	977.1	938	965.9	908	954.5
	PSD	553.2	564.5	519.7	535.6	569.8	589.2	546.9	579.5	527.5	562.7	517.6	532.7
ZQL17	Makespan	866	885.8	844	863.5	929	940.6	858	883.3	833	868.1	811	840.3
	PSD	398.5	405.3	372.8	386.8	409.6	424.0	411.9	422.6	376.6	403.2	375.8	383.0
ZQL18	Makespan	987	1025.6	1016	1027.9	1067	1081.9	993	1026.7	999	1016.2	986	1008.4
	PSD	555.3	569.4	539.1	558.9	583.7	609.4	571.4	595.7	530.0	557.2	522.3	554.7
ZQL19	Makespan	882	910.5	914	920.4	951	966.2	890	912.9	879	892.5	886	894.5
	PSD	510.0	525.8	474.4	496.3	556.0	571.7	519.0	546.9	499.1	534.4	498.4	514.3
ZQL20	Makespan	1001	1016.9	1012	1020.9	1052	1069.4	988	1030.8	992	1011.0	978	1005.7
	PSD	487.7	507.1	498.6	508.3	524.3	542.5	513.9	532.0	503.7	521.6	485.4	500.6

of the ZQL test suite and obtains the smallest average value on 17 instances. This proves that, with equal computational resources, MCPEA has a stronger capability for obtaining optimal scheduling solutions and robustness. In individual instances, such as ZQL05, ZQL06, and ZQL19, KMA achieves better or comparable results in terms of the best makespan, demonstrating its competitiveness as a new algorithm. However, the average value of MCPEA in these instances is nearly the same as those of KMA, and even better on ZQL19, while significantly leading in the vast majority of other instances, reflecting its more comprehensive superiority.

In terms of satisfying job priority constraints, MCPEA also demonstrates overwhelmingly superior performance under the same computation time. As a key indicator for measuring an algorithm's ability to handle priority constraints, MCPEA achieves the best *PSD* value on 18 instances, which fully proves the effectiveness of our designed priority-driven segmented encoding and

decoding scheme and the multi-critical-path neighbourhood search. Even in individual instances where the makespan of MCPEA is not optimal, such as ZQL05 and ZQL06, its *PSD* value still remains optimal. This indicates that the proposed method successfully realises the optimisation goal of ensuring timely completion of high-priority jobs while pursuing production efficiency. In contrast, other algorithms show significant deficiencies in meeting this specific constraint.

Convergence curves can provide a more intuitive representation of the convergence performance of each algorithm on different instances. Therefore, Figure 11 shows the convergence graphs of MCPEA and the five comparative algorithms on several instances. It can be seen that convergence curves of MCPEA are always lower than other curves, with a faster convergence speed and higher convergence accuracy. Then, it is not difficult to find that the starting point of MCPEA's convergence curve is significantly lower than that of other algorithms.

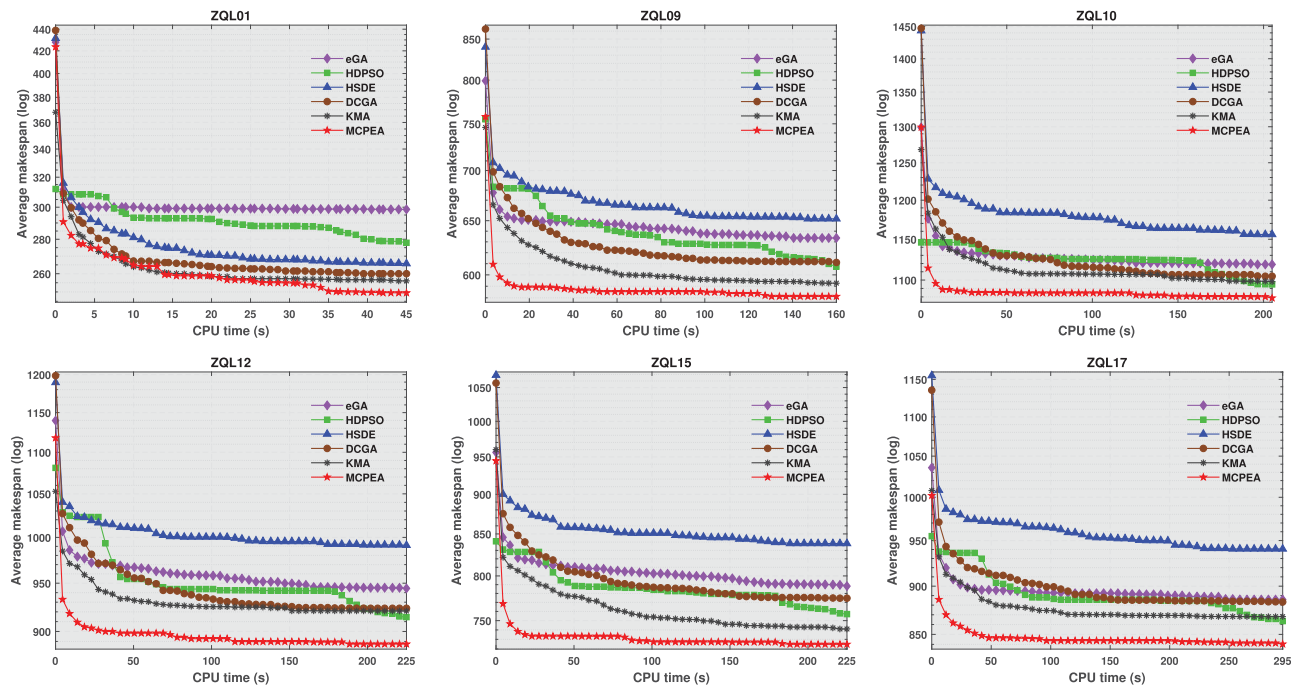


Figure 11. Convergence curves of eGA, HDPSO, HSDE, DCGA, KMA and MCPEA on some benchmark instances.

This demonstrates that our hybrid heuristic initialisation method significantly enhances the quality of the first generation population. Furthermore, the rapid decline of MCPEA’s convergence curve in the early iterations can be attributed to the migration operator based on exemplar selection, which effectively explores better solutions within the problem solution space, thereby improving the convergence speed. In the middle and later stages of evolution, the neighbourhood structure co-driven by multi-critical-path plays a role and continuously exploits approximate optimal solutions.

To obtain scientifically valid statistical results, we conducted the Friedman test on the experimental data (the mean value of makespan) presented in Table 9 to perform an overall comparison of the six algorithms. The results are shown in Table 10, where ‘Count’ indicates the number of times the algorithm achieves the best mean value on 20 instances, ‘Avg.rank’ represents the average ranking of the algorithm, and ‘Total.rank’ indicates the final ranking. From Table 10, we can see that the average rankings for eGA, HDPSO, HSDE, DCGA, KMA, and MCPEA are 4.60, 3.55, 5.80, 3.80, 2.05, and 1.15, respectively. Thus, among the six algorithms, the proposed MCPEA ranks the highest and performs the best overall. Following MCPEA, KMA ranks second, DCGA ranks third, and HSDE ranks last. Building on this, Figure 12 presents the radar chart displaying the rankings of the six algorithms across various instances. Obviously, except for three instances where our algorithm ranks lower than KMA, MCPEA occupies the top spot

Table 10. The Friedman test results of Table 9.

Index	eGA	HDPSO	HSDE	DCGA	KMA	MCPEA
Count	0	0	0	0	3	17
Ave.rank	4.60	3.55	5.80	3.80	2.05	1.15
Total.rank	5	4	6	3	2	1

on the remaining instances. This statistical conclusion confirms the performance of MCPEA in optimising the makespan.

In summary, the systematic experimental results indicate that the proposed MCPEA not only demonstrates significant advantage on the traditional production efficiency metric (makespan) when solving the DRFJSP-OJP, but also exhibits superior performance in meeting the job priority constraints that are essential in practical production. The overall framework of MCPEA and its core components are effective and advanced for addressing this complex problem.

6. Case study

6.1. Application of the algorithm in a practical case

The proposed algorithm is applied in this section to a real case in a workshop of one complex structural components manufacturing enterprise. This enterprise mainly undertakes the manufacturing of structural components for some high-end equipment, such as aircraft engine blades, optical mirror mounts, and electrostatic chucks. The machining line in the studied workshop

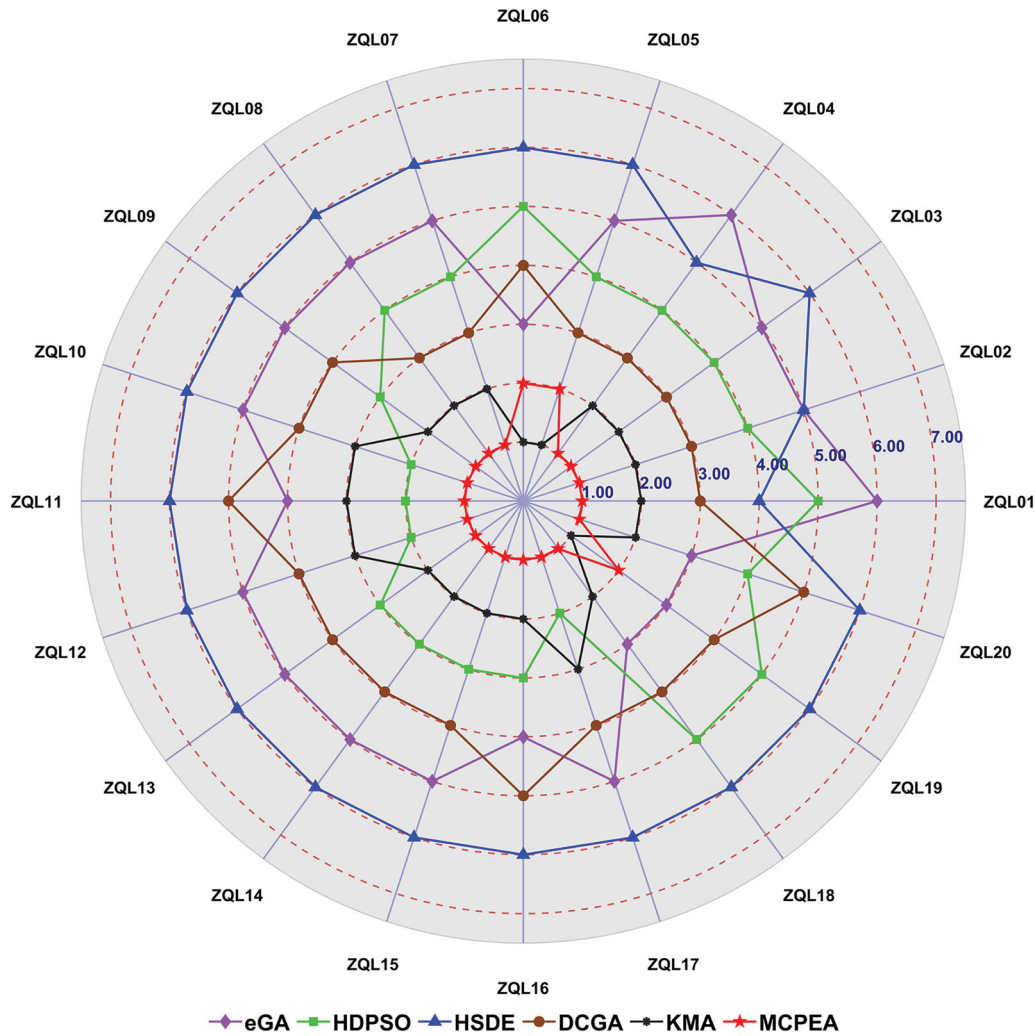


Figure 12. The radar chart of eGA, HDPSO, HSDE, DCGA, KMA and MCPEA on 20 benchmark instances.

has eight machines capable of performing processes such as turning, milling, drilling, boring, and grinding. These machines are all semi-automated and workers must be assigned to collaborate with them for processing. The manufacturing of complex structural components involves not only online operations that must be processed on the machining line, but also offline operations such as aging, sandblasting, marking, and deburring. These operations need to be performed away from the machining line on fixed workstations. For instance, aging is a heat treatment operation where workers place the workpiece into the box-type aging furnace for heating, holding, and cooling. Therefore, online operations must be completed by workers on the machines, while offline operations can be carried out independently by workers. Table 11 provides detailed information about this actual case, including nine jobs, eight machines, and seven workers. In addition, each production task has different priorities, release times, and delivery times. It can be seen that the resource allocation in this workshop is

particularly complex, with strong coupling relationships among tasks, machines and workers. Currently, formulating an efficient production plan is a major challenge that this enterprise urgently needs to address.

Table 12 shows information on the skills of workers and the types of machines. Among them, workers W_1 , W_2 , W_5 , and W_6 can both perform online and offline operations, which means they need to frequently leave or return to the machining line. In contrast, workers W_3 , W_4 , and W_7 can only collaborate with machines on the machining line for processing. At the same time, the more processes a machine can perform, the higher the skill level required for the workers. For example, machine M_7 is a multi-tasking machining center that can execute various processes, and currently, only worker W_4 is able to operate it. While M_1 can only perform turning, and workers W_1 , W_2 , W_4 , and W_5 all possess the corresponding skill to operate it.

Before applying the proposed algorithm, this workshop used a scheduling method based on the workers'

Table 11. Detailed information for the actual case from the workshop, including nine jobs, eight machines and seven workers.

Jobs	Priority	Release time	Delivery time	Operation Number	Process name	Operation Type	Machine/Worker set	Processing time
J_1	2	17	102	O_{11}	Turning	Online	M_1, M_2, M_7	31, 32, 34
				O_{12}	Milling	Online	M_3, M_5, M_7	40, 38, 41
				O_{13}	Turning	Online	M_1, M_2, M_7	28, 25, 30
				O_{14}	Aging	Offline	W_1, W_2, W_5, W_6	12
				O_{15}	Drilling	Online	M_4, M_7	18, 20
				O_{16}	Boring	Online	M_7, M_8	26, 23
				O_{17}	Deburring	Offline	W_2, W_5, W_6	8
J_2	1	23	158	O_{21}	Milling	Online	M_3, M_5, M_7	25, 27, 28
				O_{22}	Turning	Online	M_1, M_2, M_7	31, 28, 33
				O_{23}	Aging	Offline	W_1, W_2, W_5, W_6	15
				O_{24}	Boring	Online	M_7, M_8	20, 17
				O_{25}	Blasting	Offline	W_1, W_2	9
				O_{26}	Grinding	Online	M_6, M_7	22, 24
				O_{27}	Marking	Offline	W_5, W_6	12
J_3	1	10	246	O_{31}	Turning	Online	M_1, M_2, M_7	35, 37, 38
				O_{32}	Milling	Online	M_3, M_5, M_7	25, 28, 29
				O_{33}	Aging	Offline	W_1, W_2, W_5, W_6	22
				O_{34}	Drilling	Online	M_4, M_7	38, 41
				O_{35}	Milling	Online	M_3, M_5, M_7	29, 26, 29
				O_{36}	Deburring	Offline	W_2, W_5, W_6	19
J_4	3	0	188	O_{41}	Turning	Online	M_1, M_2, M_7	29, 30, 32
				O_{42}	Milling	Online	M_3, M_5, M_7	26, 28, 28
				O_{43}	Aging	Offline	W_1, W_2, W_5, W_6	16
				O_{44}	Boring	Online	M_7, M_8	18, 18
J_5	3	6	275	O_{51}	Turning	Online	M_1, M_2, M_7	24, 24, 25
				O_{52}	Drilling	Online	M_4, M_7	32, 34
				O_{53}	Milling	Online	M_3, M_5, M_7	27, 25, 28
				O_{54}	Boring	Online	M_7, M_8	23, 19
				O_{55}	Blasting	Offline	W_1, W_2	15
				O_{56}	Marking	Offline	W_5, W_6	12
J_6	1	16	260	O_{61}	Turning	Online	M_1, M_2, M_7	24, 22, 25
				O_{62}	Milling	Online	M_3, M_5, M_7	26, 26, 30
				O_{63}	Aging	Offline	W_1, W_2, W_5, W_6	18
				O_{64}	Boring	Online	M_7, M_8	21, 18
				O_{65}	Drilling	Online	M_4, M_7	20, 22
				O_{66}	Deburring	Offline	W_2, W_5, W_6	18
				O_{67}	Blasting	Offline	W_1, W_2	13
				O_{68}	Grinding	Online	M_6, M_7	23, 25
J_7	2	9	324	O_{71}	Milling	Online	M_3, M_5, M_7	42, 38, 44
				O_{72}	Turning	Online	M_1, M_2, M_7	27, 27, 29
				O_{73}	Aging	Offline	W_1, W_2, W_5, W_6	20
				O_{74}	Drilling	Online	M_4, M_7	34, 36
				O_{75}	Grinding	Online	M_6, M_7	37, 40
				O_{76}	Marking	Offline	W_5, W_6	16
J_8	3	12	295	O_{81}	Turning	Online	M_1, M_2, M_7	32, 33, 35
				O_{82}	Milling	Online	M_3, M_5, M_7	27, 24, 28
				O_{83}	Boring	Online	M_7, M_8	18, 17
				O_{84}	Blasting	Offline	W_1, W_2	22
				O_{85}	Marking	Offline	W_5, W_6	14
J_9	1	21	212	O_{91}	Milling	Online	M_3, M_5, M_7	26, 28, 30
				O_{92}	Aging	Offline	W_1, W_2, W_5, W_6	11
				O_{93}	Boring	Online	M_7, M_8	27, 24
				O_{94}	Drilling	Online	M_4, M_7	28, 32
				O_{95}	Deburring	Offline	W_2, W_5, W_6	9
				O_{96}	Blasting	Offline	W_1, W_2	15
				O_{97}	Grinding	Online	M_6, M_7	18, 23

experience. For example, the longest processing time (LPT) rule is used to determine the operation sequence. At the same time, the minimum workload (MW) rule is used to prioritise machines and workers with the least workload to allocate resources. For the actual case in Table 11, the production cycle of the enterprise's original scheduling plan is 623, as shown in Figure 13. This

scheme results in a 77.77% delay rate, with a low priority satisfaction degree and longer idle time for the machines. The optimised scheduling scheme, using the algorithm proposed in this paper, is shown in Figure 14. The new plan shortens the production cycle to 314, which is 49.60% lower than the original scheme, and the delay rate is reduced by 33.33%. Meanwhile, the preferentially

Table 12. The specific information of the workers and the machines for this case.

Workers	Skills	Machines	Mode
W_1	Turning, Boring, Aging, and Blasting	M_1, M_2 (Turning), and M_8 (Boring)	Dual-mode
W_2	Turning, Grinding, Aging, Blasting, and Deburring	M_1, M_2 (Turning), and M_6 (Grinding)	Dual-mode
W_3	Milling, and Boring	M_3, M_5 (Milling), and M_8 (Boring)	Single-mode
W_4	Turning, Milling, Drilling, Grinding, and Boring	M_1, M_2 (Turning), M_3, M_5 (Milling), M_4 (Drilling), M_6 (Grinding), M_8 (Boring), and M_7 (Turning, Milling, Drilling, Grinding, Boring)	Single-mode
W_5	Turning, Aging, Marking, and Deburring	M_1, M_2 (Turning)	Dual-mode
W_6	Milling, Drilling, Aging, Marking, and Deburring	M_3, M_5 (Milling), and M_4 (Drilling)	Dual-mode
W_7	Milling, and Boring	M_3, M_5 (Milling), and M_8 (Boring)	Single-mode

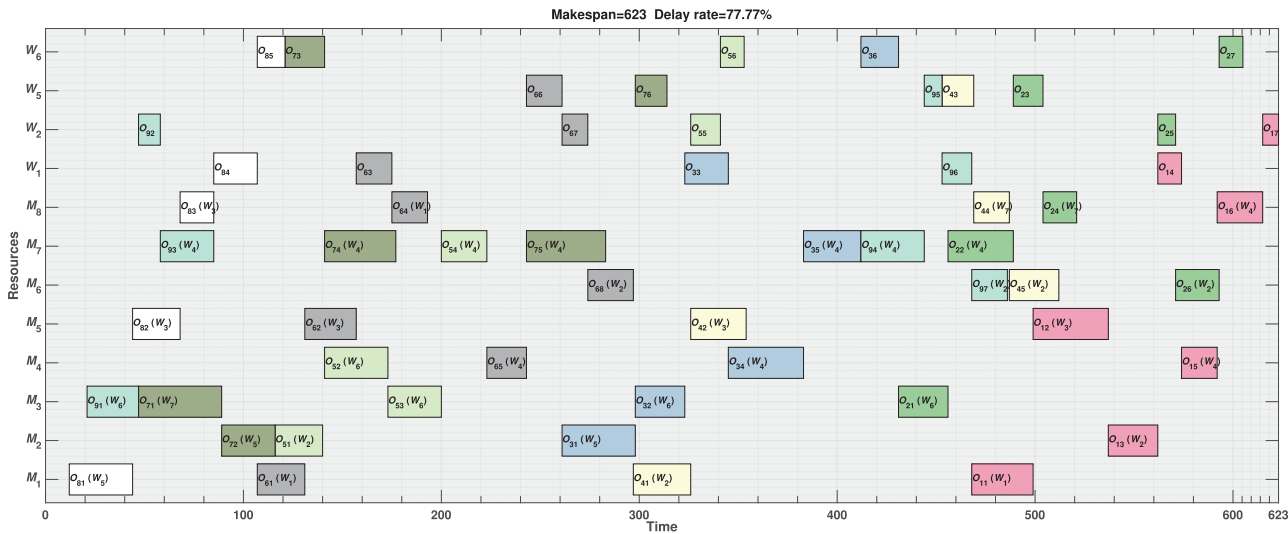


Figure 13. The gantt chart of the original scheduling scheme of the enterprise ($C_{\max} = 623$).

production of high-priority orders such as J_4 , J_5 and J_8 is guaranteed, and their completion times are generally earlier than other low-priority tasks. Moreover, the equipment utilisation rate has also been improved, especially with significant reductions in idle times for M_1 , M_3 ,

and M_4 . Therefore, the results of this case study indicate that the proposed MCPEA can effectively shorten the production cycle of the workshop and ensure the early completion of high-priority tasks, demonstrating practical application value.

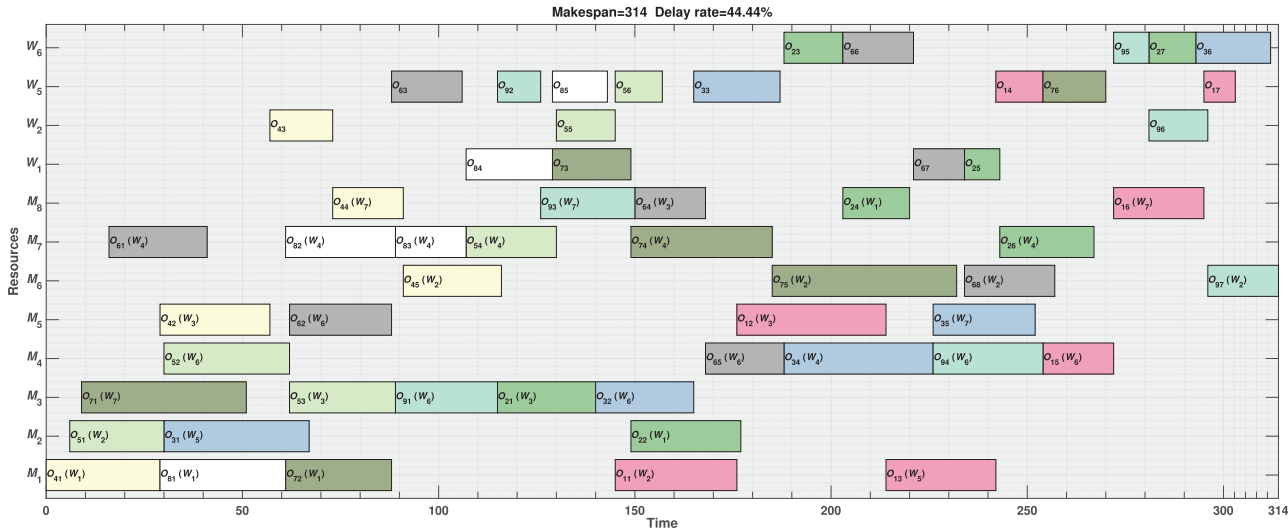


Figure 14. The gantt chart of the optimal scheduling scheme from the proposed algorithm ($C_{\max} = 314$).

6.2. Management insights

Through systematic experimental analysis and the case study, this subsection extracts the following management insights to provide decision support for production planning and scheduling in manufacturing enterprises, particularly in the high-customisation, multi-variety, and small-batch production environment.

- (1) Managers should emphasise the collaborative scheduling of machines and workers, especially in complex workshops with offline operations. Reasonable allocation of workers effectively alleviates the pressure on bottleneck resources. This requires enterprises to integrate the skills, patterns, and flexibility of human resources as important modules when building or upgrading the manufacturing system.
- (2) This study confirms that embedding job priorities into the scheduling system ensures the timely completion of high-value orders. Managers should establish a quantitative priority assessment system based on factors such as profit, penalties, customer ratings, and strategic importance, and use corresponding algorithms to convert these priorities into executable scheduling plans, achieving a transition from 'pursuing maximum efficiency' to 'ensuring maximum value'.
- (3) For workers with comprehensive skills who can operate multiple machines, or for equipment capable of performing various operations, they should not be allocated at the initial stage of scheduling. Instead, managers should treat these high-flexibility resources as 'strategic substitutes', prioritising the allocation of dedicated resources in the early planning stages while reserving high-flexibility resources for later stages to fill resource gaps or address bottleneck operations. For example, in the aforementioned case, M_7 and W_4 can serve as alternative options for the allocation of machines and workers, thereby reducing the occurrence of bottleneck resource situations.

7. Conclusion and future work

This paper defines the dual-resource flexible job shop scheduling problem with offline operations and job priority constraints. The problem considers both the operations that do not require processing on the machining line and the priority constraints of different jobs, making it more aligned with actual production background than previous studies. To address this issue, a MILP model is first developed to accurately map all constraints. Based on this, we propose a

multi-critical-path co-driven evolutionary algorithm, called MCPEA. In this algorithm, a priority-driven three-layer segmented encoding scheme and priority-based multi-segment active decoding scheme are designed. Then, a migration operator based on exemplar selection is introduced to strengthen information exchange among population individuals, improving the algorithm's convergence speed. Furthermore, the unique global critical-path and multiple local critical-paths of DRFJSP-OJP are defined, and a problem-specific neighbourhood structure that satisfies resource coupling constraints is designed for common critical operations.

In numerical experiments, an ablation study is first conducted to thoroughly evaluate the effectiveness of both the proposed hybrid heuristic initialisation, evolutionary operators and the local search strategy. Then, we validate the correctness of the developed MILP model, and compare the performance of the proposed algorithm with several advanced algorithms from recent years. Experimental results show that the MILP model can obtain feasible solutions for small-scale instances within a limited time. In contrast, MCPEA demonstrates superior convergence and stability, outperforming other algorithms in both makespan minimisation and priority satisfaction. Finally, the proposed method is implemented in a real case involving a complex structural component manufacturing enterprise, achieving a 49.60% reduction in the original makespan and a 33.33% decrease in the delay rate.

This study primarily focuses on the static scheduling scenario, while future work can delve into algorithm design in the dynamic scheduling environment. Future work will address dynamic scheduling scenarios by incorporating real-world uncertainties such as machine failures, rush orders, and worker unavailability. This will require designing responsive mechanisms to handle such disruptions effectively. Based on the proposed MCPEA, the focus will be on breaking through the collaborative optimisation problem of priority constraints and dynamic disturbances, developing an adaptive scheduling system with online response capabilities, and further enhancing the practical value of the algorithm in complex industrial scenarios.

Acknowledgements

The authors would like to express their sincere thanks to the editors and reviewers for their valuable suggestions and comments.

Disclosure statement

The authors declare that there are no competing interests regarding the publication of this manuscript. All authors have

disclosed any financial or personal relationships that could be perceived as potential conflicts of interest. The research was conducted independently, and the findings presented in this paper reflect the authors' views and interpretations. There are no affiliations or financial arrangements that could influence the results or conclusions of this study.

Author contributions

Ziyu Zhang: Conceptualization, Methodology, Software, Formal analysis, Writing – original draft. **Dongchen Qiu:** Validation, Resources, Project Administration. **Xinyu Li:** Supervision, Funding acquisition, Writing – review. **Liang Gao:** Investigation, Supervision, Writing – review. **Qihao Liu:** Visualization, Funding acquisition, Writing – review & editing. **Yue Teng:** Data curation, Writing – Review & Editing. **Xuxia Zhang:** Resources. **Jun Wu:** Resources.

Funding

This work was supported by the National Natural Science Foundation of China under Grant 52305534, the Key Research and Development Program of Zhejiang Province under Grant 2024C01139, and the Fundamental Research Funds for the Central Universities under Grant 2024BRA004.

Notes on contributors



Ziyu Zhang He is currently pursuing the Ph.D. degree in mechanical engineering with the Huazhong University of Science and Technology (HUST), Wuhan, China. His current research interests include evolutionary computation, machine learning, and their applications in production scheduling.



Dongchen Qiu received the B.Eng. degree from the Nanjing Tech University, Nanjing, China, in 2011. He currently works as a technician in the Production and Manufacturing Department of China Tobacco Jiangsu Industrial Co., Ltd.. His current research focuses on the planning and scheduling of automated tobacco production processes.



Xinyu Li received the Ph.D. degree in industrial engineering from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2009. He is currently a Professor with the Department of Industrial and Manufacturing Systems Engineering, State Key Laboratory of Intelligent Manufacturing Equipment and Technology, School of Mechanical Science and Engineering, HUST. He has published more than 100 refereed articles. His research interests include intelligent manufacturing system, big data, and machine learning.



Liang Gao received the B.Sc. degree in mechatronic engineering from the Xidian University, Xi'an, China, in 1996, and the Ph.D. degree in mechatronic engineering from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2002. He is currently a Professor with the Department of Industrial and Manufacturing System Engineering, State Key Laboratory of Intelligent Manufacturing Equipment and Technology, School of Mechanical Science and Engineering, HUST. He has published more than 400 refereed articles. His research interests include operations research and optimization, big data, and machine learning. Prof. Gao currently serves as the Co-Editor-in-Chief for IET Collaborative Intelligent Manufacturing and an Associate Editor for Swarm and Evolutionary Computation and Chinese Journal of Mechanical Engineering.



Qihao Liu received the Ph.D. degree in industrial engineering from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2022. From 2022 to 2024, he was a Postdoctoral Researcher with HUST. He is currently a Lecturer of industrial engineering with the School of Mechanical Science and Engineering, HUST. His research interests include process planning and shop scheduling.



Yue Teng received the B.Eng. degree in industrial engineering from the Huazhong University of Science and Technology (HUST), Wuhan, China, 2020. He is currently pursuing the Ph.D. degree in mechanical engineering with HUST. His current research interests include discrete optimisation and shop scheduling.



Xuxia Zhang She is a senior engineer and serves as the Chief Engineer of Zhejiang Wanfeng Technology Development Co., Ltd., Shaoxing, China. Her achievements include three national project honors, 15 invention patents, 67 utility model patents, five software copyrights, 13 published papers, and she has spearheaded the development of 11 national industry standards. Her current research focus on intelligent control systems and casting management systems.



Jun Wu He is a senior engineer and serves as the General Manager of Zhejiang Wanfeng Technology Development Co., Ltd., Shaoxing, China. His achievements include three national project honors, 21 provincial honors, two horizontal research projects, 18 invention patents, and participation in the formulation of two national industry standards. His current research focuses on the development and application of intelligent casting production lines.

Data availability statement

The dataset of the ZQL test suite is publicly available at <https://zenodo.org/records/17226700>. The MATLAB code is available from the corresponding author on reasonable request.

ORCID

Ziyu Zhang  <http://orcid.org/0000-0002-4140-6467>

Xinyu Li  <http://orcid.org/0000-0002-3730-0360>

References

- Ahmadi-Javid, A., and P. Hooshangi-Tabrizi. 2017. "Integrating Employee Timetabling with Scheduling of Machines and Transporters in a Job-Shop Environment: A Mathematical Formulation and an Anarchic Society Optimization Algorithm." *Computers & Operations Research* 84:73–91. <https://doi.org/10.1016/j.cor.2016.11.017>.
- Andrade-Pineda, J. L., D. Canca, P. L. Gonzalez-R, and M. Calle. 2020. "Scheduling a Dual-Resource Flexible Job Shop with Makespan and Due Date-Related Criteria." *Annals of Operations Research* 291 (1-2): 5–35. <https://doi.org/10.1007/s10479-019-03196-0>.
- Bakon, K., T. Holczinger, Z. Süle, S. Jaskó, and J. Abonyi. 2022. "Scheduling under Uncertainty for Industry 4.0 and 5.0." *IEEE Access* 10:74977–75017. <https://doi.org/10.1109/ACCESS.2022.3191426>.
- Brucker, P., and R. Schlie. 1990. "Job-Shop Scheduling with Multi-purpose Machines." *Computing* 45 (4): 369–375. <https://doi.org/10.1007/BF02238804>.
- Caldeira, R. H., and A. Gnanavelbabu. 2021. "A Simheuristic Approach for the Flexible Job Shop Scheduling Problem with Stochastic Processing times." *Simulation* 97:215–236. <https://doi.org/10.1177/0037549720968891>.
- Chen, C. A., H. A. Kuo, and C. F. Chien. 2024. "Dual-Resource Constrained Flexible Job Shop Scheduling for Smt Back-End Production and an Empirical Study of Wearable Devices." *IEEE Transactions on Automation Science and Engineering* 22:9230–9239. <https://doi.org/10.1109/TASE.2024.3503412>.
- Chen, R., B. Yang, S. Li, and S. Wang. 2020. "A Self-learning Genetic Algorithm Based on Reinforcement Learning for Flexible Job-Shop Scheduling Problem." *Computers & Industrial Engineering* 149:106778. <https://doi.org/10.1016/j.cie.2020.106778>.
- Chen, Y., R. Goebel, G. Lin, B. Su, and A. Zhang. 2020. "Open-Shop Scheduling for Unit Jobs under Precedence Constraints." *Theoretical Computer Science* 803:144–151. <https://doi.org/10.1016/j.tcs.2019.09.046>.
- Dauzère-Pérès, S., J. Ding, L. Shen, and K. Tamssauet. 2024. "The Flexible Job Shop Scheduling Problem: A Review." *European Journal of Operational Research* 314 (2): 409–432. <https://doi.org/10.1016/j.ejor.2023.05.017>.
- Delgoshaei, A., M. K. A. M. Ariffin, S. Maleki, and Z. Leman. 2023. "Review Evolution of Dual-Resource-constrained Scheduling Problems in Manufacturing Systems: Modeling and Scheduling Methods' Trends." *Soft Computing-A Fusion of Foundations, Methodologies & Applications* 27:18489–18528.
- Deng, L., Y. Qiu, Y. Di, and L. Zhang. 2025. "A Knowledge-Driven Memetic Algorithm for Distributed Green Flexible Job Shop Scheduling considering the Endurance of Machines." *Applied Soft Computing* 170:112697. <https://doi.org/10.1016/j.asoc.2025.112697>.
- Destouet, C., H. Tlahig, B. Bettayeb, and B. Mazari. 2023. "Flexible Job Shop Scheduling Problem under Industry 5.0: a Survey on Human Reintegration, Environmental Consideration and Resilience Improvement." *Journal of Manufacturing Systems* 67:155–173. <https://doi.org/10.1016/j.jmsy.2023.01.004>.
- Fan, J., W. Shen, L. Gao, C. Zhang, and Z. Zhang. 2021. "A Hybrid Jaya Algorithm for Solving Flexible Job Shop Scheduling Problem considering Multiple Critical Paths." *Journal of Manufacturing Systems* 60:298–311. <https://doi.org/10.1016/j.jmsy.2021.05.018>.
- Fattahi, P., S. M. H. Hosseini, F. Jolai, and R. Tavakkoli-Moghaddam. 2014. "A Branch and Bound Algorithm for Hybrid Flow Shop Scheduling Problem with Setup Time and Assembly Operations." *Applied Mathematical Modelling* 38 (1): 119–134. <https://doi.org/10.1016/j.apm.2013.06.005>.
- Frihat, M., A. B. Hadj-Alouane, and C. Sadfi. 2022. "Optimization of the Integrated Problem of Employee Timetabling and Job Shop Scheduling." *Computers & Operations Research* 137:105332. <https://doi.org/10.1016/j.cor.2021.105332>.
- Gao, K., Z. Cao, L. Zhang, Z. Chen, Y. Han, and Q. Pan. 2019. "A Review on Swarm Intelligence and Evolutionary Algorithms for Solving Flexible Job Shop Scheduling Problems." *IEEE/CAA Journal of Automatica Sinica* 6:904–916. <https://doi.org/10.1109/JAS.2019.1911540>.
- Gao, K. Z., P. N. Suganthan, Q. K. Pan, T. J. Chua, T. X. Cai, and C. S. Chong. 2016. "Discrete Harmony Search Algorithm for Flexible Job Shop Scheduling Problem with Multiple Objectives." *Journal of Intelligent Manufacturing* 27 (2): 363–374. <https://doi.org/10.1007/s10845-014-0869-8>.
- Gnanavelbabu, A., R. H. Caldeira, and T. Vaidyanathan. 2021. "A Simulation-Based Modified Backtracking Search Algorithm for Multi-objective Stochastic Flexible Job Shop Scheduling Problem with Worker Flexibility." *Applied Soft Computing* 113:107960. <https://doi.org/10.1016/j.asoc.2021.107960>.
- Goli, A., E. B. Tirkolaee, and N. S. Aydın. 2021. "Fuzzy Integrated Cell Formation and Production Scheduling considering Automated Guided Vehicles and Human Factors." *IEEE Transactions on Fuzzy Systems* 29:3686–3695. <https://doi.org/10.1109/TFUZZ.2021.3053838>.
- Gong, G., R. Chiong, Q. Deng, and X. Gong. 2020. "A Hybrid Artificial Bee Colony Algorithm for Flexible Job Shop Scheduling with Worker Flexibility." *International Journal of Production Research* 58 (14): 4406–4420. <https://doi.org/10.1080/00207543.2019.1653504>.
- Gong, G., Q. Deng, X. Gong, and D. Huang. 2021. "A Non-dominated Ensemble Fitness Ranking Algorithm for Multi-objective Flexible Job-Shop Scheduling Problem considering Worker Flexibility and Green Factors." *Knowledge-Based Systems* 231:107430. <https://doi.org/10.1016/j.knsys.2021.107430>.
- Gong, G., Q. Deng, X. Gong, W. Liu, and Q. Ren. 2018. "A New Double Flexible Job-Shop Scheduling Problem Integrating Processing Time, Green Production, and Human Factor Indicators." *Journal of Cleaner Production* 174:560–576. <https://doi.org/10.1016/j.jclepro.2017.10.188>.
- Gong, X., Q. Deng, G. Gong, W. Liu, and Q. Ren. 2018. "A Memetic Algorithm for Multi-objective Flexible Job-Shop Problem with Worker Flexibility." *International Journal of*

- Production Research* 56 (7): 2506–2522. <https://doi.org/10.1080/00207543.2017.1388933>.
- Hajariwala, D. C., S. S. Patil, and S. M. Patil. 2025. “A Review of Metaheuristic Algorithms for Job Shop Scheduling.” *Engineering Access* 11:65–91.
- Ham, A. 2017. “Flexible Job Shop Scheduling Problem for Parallel Batch Processing Machine with Compatible Job Families.” *Applied Mathematical Modelling* 45:551–562. <https://doi.org/10.1016/j.apm.2016.12.034>.
- Han, K., and W. Gong. 2025. “Memetic Algorithm Based on Non-dominated Levels for Flexible Job Shop Scheduling Problem with Learn-Forgetting Effect and Worker Cooperation.” *Computers & Industrial Engineering* 200:110845. <https://doi.org/10.1016/j.cie.2024.110845>.
- Han, X., W. Cheng, L. Meng, B. Zhang, K. Gao, C. Zhang, and P. Duan. 2024. “A Dual Population Collaborative Genetic Algorithm for Solving Flexible Job Shop Scheduling Problem with Agv.” *Swarm and Evolutionary Computation* 86:101538. <https://doi.org/10.1016/j.swevo.2024.101538>.
- He, Z., B. Tang, and F. Luan. 2022. “An Improved African Vulture Optimization Algorithm for Dual-Resource Constrained Multi-objective Flexible Job Shop Scheduling Problems.” *Sensors* 23:90. <https://doi.org/10.3390/s23010090>.
- Hongyu, L., and W. Xiuli. 2021. “A Survival Duration-Guided Nsga-Iii for Sustainable Flexible Job Shop Scheduling Problem considering Dual Resources.” *IET Collaborative Intelligent Manufacturing* 3 (2): 119–130. <https://doi.org/10.1049/cim2.v3.2>.
- Huang, J., X. Li, and L. Gao. 2025. “A Novel Ga-Cp Method for Fixed-Type Multi-robot Collaborative Scheduling in Flexible Job Shop.” *IEEE Transactions on Automation Science and Engineering* 22:13531–13543. <https://doi.org/10.1109/TASE.2025.3554019>.
- Huang, K., W. Gong, and C. Lu. 2024. “An Enhanced Memetic Algorithm with Hierarchical Heuristic Neighborhood Search for Type-2 Green Fuzzy Flexible Job Shop Scheduling.” *Engineering Applications of Artificial Intelligence* 130:107762. <https://doi.org/10.1016/j.engappai.2023.107762>.
- Huang, L., S. Zhao, and Q. Han. 2022. “A Fast Layered Path Planning Algorithm for Job Shop Scheduling Problem.” *IET Collaborative Intelligent Manufacturing* 4 (4): 299–315. <https://doi.org/10.1049/cim2.v4.4>.
- Huang, X., Z. Guan, and L. Yang. 2018. “An Effective Hybrid Algorithm for Multi-objective Flexible Job-Shop Scheduling Problem.” *Advances in Mechanical Engineering* 10:1–14.
- Kaban, A., Z. Othman, and D. Rohmah. 2012. “Comparison of Dispatching Rules in Job-Shop Scheduling Problem Using Simulation: A Case Study.” *International Journal of Simulation Modelling* 11 (3): 129–140. <https://doi.org/10.2507/IJ SIMM>.
- Kim, T. Y. 2020. “Improving Warehouse Responsiveness by Job Priority Management: A European Distribution Centre Field Study.” *Computers & Industrial Engineering* 139:105564. <https://doi.org/10.1016/j.cie.2018.12.011>.
- Lei, K., P. Guo, W. Zhao, Y. Wang, L. Qian, X. Meng, and L. Tang. 2022. “A Multi-action Deep Reinforcement Learning Framework for Flexible Job-Shop Scheduling Problem.” *Expert Systems with Applications* 205:117796. <https://doi.org/10.1016/j.eswa.2022.117796>.
- Li, H., X. Li, and L. Gao. 2024. “An Iterated Greedy Algorithm with Acceleration of Job Allocation Probability for Distributed Heterogeneous Permutation Flowshop Scheduling Problem.” *Swarm and Evolutionary Computation* 88:101580. <https://doi.org/10.1016/j.swevo.2024.101580>.
- Li, H., J. Peng, and X. Wang. 2024. “An Efficient Two-Stage Optimization Algorithm for a Flexible Job Shop Scheduling Problem with Worker Shift Arrangement.” *Computers & Operations Research* 171:106785. <https://doi.org/10.1016/j.cor.2024.106785>.
- Li, H., X. Wang, and J. Peng. 2022. “A Hybrid Differential Evolution Algorithm for Flexible Job Shop Scheduling with Outsourcing Operations and Job Priority Constraints.” *Expert Systems with Applications* 201:117182. <https://doi.org/10.1016/j.eswa.2022.117182>.
- Li, J., Q. Liu, X. Li, and L. Gao. 2025. “An Efficient Problem-Specific Evolutionary Algorithm for Flexible Job Shop Scheduling Problem with Specific Workers in Highly Customised Manufacturing Systems.” *International Journal of Production Research* 63 (19): 7238–7259. <https://doi.org/10.1080/00207543.2025.2496971>.
- Li, M., and G. G. Wang. 2022. “A Review of Green Shop Scheduling Problem.” *Information Sciences* 589:478–496. <https://doi.org/10.1016/j.ins.2021.12.122>.
- Li, X., and L. Gao. 2016. “An Effective Hybrid Genetic Algorithm and Tabu Search for Flexible Job Shop Scheduling Problem.” *International Journal of Production Economics* 174:93–110. <https://doi.org/10.1016/j.ijpe.2016.01.016>.
- Li, Y., X. Chen, Y. An, Z. Zhao, H. Cao, and J. Jiang. 2023. “Integrating Machine Layout, Transporter Allocation and Worker Assignment into Job-Shop Scheduling Solved by an Improved Non-dominated Sorting Genetic Algorithm.” *Computers & Industrial Engineering* 179:109169. <https://doi.org/10.1016/j.cie.2023.109169>.
- Li, Y., X. Li, L. Gao, and Z. Lu. 2025. “Multi-agent Deep Reinforcement Learning for Dynamic Reconfigurable Shop Scheduling considering Batch Processing and Worker Cooperation.” *Robotics and Computer-Integrated Manufacturing* 91:102834. <https://doi.org/10.1016/j.rcim.2024.102834>.
- Lim, C. H., and S. K. Moon. 2023. “A Two-Phase Iterative Mathematical Programming-Based Heuristic for a Flexible Job Shop Scheduling Problem with Transportation.” *Applied Sciences* 13:5215. <https://doi.org/10.3390/app13085215>.
- Liu, C., Y. Yao, and H. Zhu. 2021. “Hybrid Salp Swarm Algorithm for Solving the Green Scheduling Problem in a Double-Flexible Job Shop.” *Applied Sciences* 12:205. <https://doi.org/10.3390/app12010205>.
- Lopes, M. J. P., and J. V. de Carvalho. 2007. “A Branch-and-price Algorithm for Scheduling Parallel Machines with Sequence Dependent Setup times.” *European Journal of Operational Research* 176 (3): 1508–1527. <https://doi.org/10.1016/j.ejor.2005.11.001>.
- Lou, H., X. Wang, Z. Dong, and Y. Yang. 2022. “Memetic Algorithm Based on Learning and Decomposition for Multi-objective Flexible Job Shop Scheduling considering Human Factors.” *Swarm and Evolutionary Computation* 75:101204. <https://doi.org/10.1016/j.swevo.2022.101204>.
- Luo, C., W. Gong, and C. Lu. 2024. “Knowledge-Driven Two-Stage Memetic Algorithm for Energy-Efficient Flexible Job Shop Scheduling with Machine Breakdowns.” *Expert Systems with Applications* 235:121149. <https://doi.org/10.1016/j.eswa.2023.121149>.
- Luo, C., X. Li, W. Gong, and L. Gao. 2025. “Affinity Propagation Hierarchical Memetic Algorithm for Multimodal

- Multi-objective Flexible Job Shop Scheduling with Variable Speed.” *IEEE Transactions on Evolutionary Computation* 29:2729–2741. <https://doi.org/10.1109/TEVC.2024.3521585>.
- Luo, Q., Q. Deng, G. Gong, X. Guo, and X. Liu. 2022. “A Distributed Flexible Job Shop Scheduling Problem considering Worker Arrangement Using an Improved Memetic Algorithm.” *Expert Systems with Applications* 207:117984. <https://doi.org/10.1016/j.eswa.2022.117984>.
- Luo, Q., Q. Deng, G. Xie, and G. Gong. 2023. “A Pareto-Based Two-Stage Evolutionary Algorithm for Flexible Job Shop Scheduling Problem with Worker Cooperation Flexibility.” *Robotics and Computer-Integrated Manufacturing* 82:102534. <https://doi.org/10.1016/j.rcim.2023.102534>.
- Müller, D., and D. Kress. 2022. “Filter-and-fan Approaches for Scheduling Flexible Job Shops under Workforce Constraints.” *International Journal of Production Research* 60 (15): 4743–4765. <https://doi.org/10.1080/00207543.2021.1937745>.
- Ma, H. 2010. “An Analysis of the Equilibrium of Migration Models for Biogeography-Based Optimization.” *Information Sciences* 180 (18): 3444–3464. <https://doi.org/10.1016/j.ins.2010.05.035>.
- Mahmoodjanloo, M., R. Tavakkoli-Moghaddam, A. Baboli, and A. Bozorgi-Amiri. 2020. “Flexible Job Shop Scheduling Problem with Reconfigurable Machine Tools: An Improved Differential Evolution Algorithm.” *Applied Soft Computing* 94:106416. <https://doi.org/10.1016/j.asoc.2020.106416>.
- Meng, L., C. Zhang, B. Zhang, and Y. Ren. 2019. “Mathematical Modeling and Optimization of Energy-Conscious Flexible Job Shop Scheduling Problem with Worker Flexibility.” *IEEE Access* 7:68043–68059. <https://doi.org/10.1109/Access.6287639>.
- Mlekusch, J., and R. F. Hartl. 2025. “The Dual-Resource-constrained re-entrant Flexible Flow Shop a Constraint Programming Approach and a Hybrid Genetic Algorithm.” *International Journal of Production Research* 63 (5): 1803–1824. <https://doi.org/10.1080/00207543.2024.2392198>.
- Mraihi, T., O. B. Driss, and H. B. El-Haouzi. 2024. “Distributed Permutation Flow Shop Scheduling Problem with Worker Flexibility: Review, Trends and Model Proposition.” *Expert Systems with Applications* 238:121947. <https://doi.org/10.1016/j.eswa.2023.121947>.
- Neufeld, J. S., S. Schulz, and U. Buscher. 2023. “A Systematic Review of Multi-objective Hybrid Flow Shop Scheduling.” *European Journal of Operational Research* 309 (1): 1–23. <https://doi.org/10.1016/j.ejor.2022.08.009>.
- Pan, Z., L. Wang, J. Zheng, J. F. Chen, and X. Wang. 2022. “A Learning-Based Multipopulation Evolutionary Optimization for Flexible Job Shop Scheduling Problem with Finite Transportation Resources.” *IEEE Transactions on Evolutionary Computation* 27:1590–1603. <https://doi.org/10.1109/TEVC.2022.3219238>.
- Peng, Z., H. Zhang, H. Tang, Y. Feng, and W. Yin. 2022. “Research on Flexible Job-Shop Scheduling Problem in Green Sustainable Manufacturing Based on Learning Effect.” *Journal of Intelligent Manufacturing* 33 (6): 1725–1746. <https://doi.org/10.1007/s10845-020-01713-8>.
- Ren, W., Y. Yan, Y. Hu, and Y. Guan. 2022. “Joint Optimization for Dynamic Flexible Job-Shop Scheduling Problem with Transportation Time and Resource Constraints.” *International Journal of Production Research* 60 (18): 5675–5696. <https://doi.org/10.1080/00207543.2021.1968526>.
- Sara, B. Y. 2025. “Application of Order Priority Strategies in Project Scheduling: Qualitative Study of Industrial Companies.” *International Journal of Applied Management and Economics* 2:289–300.
- Seifi, C., M. Schulze, and J. Zimmermann. 2021. “A New Mathematical Formulation for a Potash-Mine Shift Scheduling Problem with a Simultaneous Assignment of Machines and Workers.” *European Journal of Operational Research* 292 (1): 27–42. <https://doi.org/10.1016/j.ejor.2020.10.007>.
- Shahvari, O., R. Logendran, and M. Tavana. 2022. “An Efficient Model-Based Branch-and-price Algorithm for Unrelated-Parallel Machine Batching and Scheduling Problems.” *Journal of Scheduling* 25 (5): 589–621. <https://doi.org/10.1007/s10951-022-00729-7>.
- Shi, J., M. Chen, Y. Ma, and F. Qiao. 2023. “A New Boredom-Aware Dual-Resource Constrained Flexible Job Shop Scheduling Problem Using a Two-Stage Multi-objective Particle Swarm Optimization Algorithm.” *Information Sciences* 643:119141. <https://doi.org/10.1016/j.ins.2023.119141>.
- Simon, D. 2008. “Biogeography-Based Optimization.” *IEEE Transactions on Evolutionary Computation* 12:702–713. <https://doi.org/10.1109/TEVC.2008.919004>.
- Tan, W., X. Yuan, J. Wang, and X. Zhang. 2021. “A Fatigue-Conscious Dual Resource Constrained Flexible Job Shop Scheduling Problem by Enhanced Nsga-Ii: An Application from Casting Workshop.” *Computers & Industrial Engineering* 160:107557. <https://doi.org/10.1016/j.cie.2021.107557>.
- Tang, J., G. Gong, N. Peng, K. Zhu, D. Huang, and Q. Luo. 2024. “An Effective Memetic Algorithm for Distributed Flexible Job Shop Scheduling Problem considering Integrated Sequencing Flexibility.” *Expert Systems with Applications* 242:122734. <https://doi.org/10.1016/j.eswa.2023.122734>.
- Thürer, M., M. Stevenson, and P. Renna. 2019. “Workload Control in Dual-Resource Constrained High-Variety Shops: An Assessment by Simulation.” *International Journal of Production Research* 57 (3): 931–947. <https://doi.org/10.1080/00207543.2018.1497313>.
- Thürer, M., H. Zhang, M. Stevenson, F. Costa, and L. Ma. 2020. “Worker Assignment in Dual Resource Constrained Assembly Job Shops with Worker Heterogeneity: An Assessment by Simulation.” *International Journal of Production Research* 58 (20): 6336–6349. <https://doi.org/10.1080/00207543.2019.1677963>.
- Usman, S., C. Lu, and G. Gao. 2024. “Flexible Job-Shop Scheduling with Limited Flexible Workers Using an Improved Multiobjective Discrete Teaching-learning Based Optimization Algorithm.” *Optimization and Engineering* 25 (3): 1237–1270. <https://doi.org/10.1007/s11081-023-09842-8>.
- Vahedi-Nouri, B., R. Tavakkoli-Moghaddam, Z. Hanzálek, and A. Dolgui. 2024. “Production Scheduling in a Reconfigurable Manufacturing System Benefiting from Human-Robot Collaboration.” *International Journal of Production Research* 62 (3): 767–783. <https://doi.org/10.1080/00207543.2023.2173503>.
- Vital-Soto, A., M. F. Baki, and A. Azab. 2023. “A Multi-objective Mathematical Model and Evolutionary Algorithm for the

- Dual-Resource Flexible Job-Shop Scheduling Problem with Sequencing Flexibility." *Flexible Services and Manufacturing Journal* 35 (3): 626–668. <https://doi.org/10.1007/s10696-022-09446-x>.
- Wang, C., D. Fan, Y. Liu, S. Ren, and J. Wang. 2025. "Dual-Resource Flexible Job Shop Scheduling considering Worker Proficiency Differences." *Computers & Operations Research* 184:107216. <https://doi.org/10.1016/j.cor.2025.107216>.
- Wang, C., M. Wei, Q. Liu, X. Zhang, and X. Li. 2025. "An Improved Adaptive Hybrid Algorithm for Solving Distributed Flexible Job Shop Scheduling Problem." *Swarm and Evolutionary Computation* 94:101873. <https://doi.org/10.1016/j.swevo.2025.101873>.
- Wang, H. 2005. "Flexible Flow Shop Scheduling: Optimum, Heuristics and Artificial Intelligence Solutions." *Expert Systems* 22:78–85. <https://doi.org/10.1111/exsy.2005.22.issue-2>.
- Wang, J., L. Wang, and X. Xiu. 2023. "A Cooperative Memetic Algorithm for Energy-Aware Distributed Welding Shop Scheduling Problem." *Engineering Applications of Artificial Intelligence* 120:105877. <https://doi.org/10.1016/j.engappai.2023.105877>.
- Wang, K., M. Guo, C. Dai, and Z. Li. 2023. "A Novel Heuristic Algorithm for Solving Engineering Optimization and Real-World Problems: People Identity Attributes-Based Information-Learning Search Optimization." *Computer Methods in Applied Mechanics and Engineering* 416:116307. <https://doi.org/10.1016/j.cma.2023.116307>.
- Wang, Z., H. Hu, and J. Gong. 2018. "Modeling Worker Competence to Advance Precast Production Scheduling Optimization." *Journal of Construction Engineering and Management* 144 (11):04018098. [https://doi.org/10.1061/\(ASCE\)CO.1943-7862.0001556](https://doi.org/10.1061/(ASCE)CO.1943-7862.0001556).
- Xie, J., L. Gao, K. Peng, X. Li, and H. Li. 2019. "Review on Flexible Job Shop Scheduling." *IET Collaborative Intelligent Manufacturing* 1 (3): 67–77. <https://doi.org/10.1049/cim2.v1.3>.
- Xie, J., X. Li, L. Gao, and L. Gui. 2023. "A New Neighbourhood Structure for Job Shop Scheduling Problems." *International Journal of Production Research* 61 (7): 2147–2161. <https://doi.org/10.1080/00207543.2022.2060772>.
- Xiong, H., S. Shi, D. Ren, and J. Hu. 2022. "A Survey of Job Shop Scheduling Problem: The Types and Models." *Computers & Operations Research* 142:105731. <https://doi.org/10.1016/j.cor.2022.105731>.
- Xu, Y., D. Wang, M. Zhang, M. Yang, and C. Liang. 2025. "Quantum Particle Swarm Optimization with Chaotic Encoding Schemes for Flexible Job-Shop Scheduling Problem." *Swarm and Evolutionary Computation* 93:101836. <https://doi.org/10.1016/j.swevo.2024.101836>.
- Xu, Y., M. Zhang, M. Yang, and D. Wang. 2024. "Hybrid Quantum Particle Swarm Optimization and Variable Neighborhood Search for Flexible Job-Shop Scheduling Problem." *Journal of Manufacturing Systems* 73:334–348. <https://doi.org/10.1016/j.jmsy.2024.02.007>.
- Yang, Z., X. Li, L. Gao, and Q. Liu. 2025. "A Novel Topological Neighborhood Structure for Flexible Job Shop Scheduling Problem with Variable Sublots." *Computers & Operations Research* 182:107120. <https://doi.org/10.1016/j.cor.2025.107120>.
- Yu, F., L. Yin, B. Zeng, C. Lu, and Z. Xiao. 2024. "A Self-learning Discrete Artificial Bee Colony Algorithm for Energy-Efficient Distributed Heterogeneous Lr Fuzzy Welding Shop Scheduling Problem." *IEEE Transactions on Fuzzy Systems* 32:3753–3764. <https://doi.org/10.1109/TFUZZ.2024.3382398>.
- Zhang, C., P. Li, Z. Guan, and Y. Rao. 2007. "A Tabu Search Algorithm with a New Neighborhood Structure for the Job Shop Scheduling Problem." *Computers & Operations Research* 34 (11): 3229–3242. <https://doi.org/10.1016/j.cor.2005.12.002>.
- Zhang, G., L. Gao, and Y. Shi. 2011. "An Effective Genetic Algorithm for the Flexible Job-Shop Scheduling Problem." *Expert Systems with Applications* 38 (4): 3563–3573. <https://doi.org/10.1016/j.eswa.2010.08.145>.
- Zhang, J., W. Wang, and X. Xu. 2017. "A Hybrid Discrete Particle Swarm Optimization for Dual-Resource Constrained Job Shop Scheduling with Resource Flexibility." *Journal of Intelligent Manufacturing* 28 (8): 1961–1972. <https://doi.org/10.1007/s10845-015-1082-0>.
- Zhang, Z., Y. Fu, K. Gao, Q. Pan, and M. Huang. 2024. "A Learning-Driven Multi-objective Cooperative Artificial Bee Colony Algorithm for Distributed Flexible Job Shop Scheduling Problems with Preventive Maintenance and Transportation Operations." *Computers & Industrial Engineering* 196:110484. <https://doi.org/10.1016/j.cie.2024.110484>.
- Zhang, Z., Y. Gao, Y. Liu, and W. Zuo. 2023. "A Hybrid Biogeography-Based Optimization Algorithm to Solve High-Dimensional Optimization Problems and Real-World Engineering Problems." *Applied Soft Computing* 144:110514. <https://doi.org/10.1016/j.asoc.2023.110514>.
- Zhang, Z., X. Li, L. Gao, Q. Liu, and J. Huang. 2025. "Tackling Dual-Resource Flexible Job Shop Scheduling Problem in the Production Line Reconfiguration Scenario: An Efficient Meta-heuristic with Critical Path-Based Neighborhood Search." *Advanced Engineering Informatics* 65:103282. <https://doi.org/10.1016/j.aei.2025.103282>.
- Zheng, X., and L. Wang. 2016. "A Knowledge-Guided Fruit Fly Optimization Algorithm for Dual Resource Constrained Flexible Job-Shop Scheduling Problem." *International Journal of Production Research* 54 (18): 5554–5566. <https://doi.org/10.1080/00207543.2016.1170226>.
- Zhou, L., Z. Jiang, N. Geng, Y. Niu, F. Cui, K. Liu, and N. Qi. 2022. "Production and Operations Management for Intelligent Manufacturing: A Systematic Literature Review." *International Journal of Production Research* 60 (2): 808–846. <https://doi.org/10.1080/00207543.2021.2017055>.
- Zhu, H., Q. Deng, L. Zhang, X. Hu, and W. Lin. 2020. "Low Carbon Flexible Job Shop Scheduling Problem considering Worker Learning Using a Memetic Algorithm." *Optimization and Engineering* 21 (4): 1691–1716. <https://doi.org/10.1007/s11081-020-09494-y>.
- Zhu, Z., X. Zhou, D. Cao, and M. Li. 2022. "A Shuffled Cellular Evolutionary Grey Wolf Optimizer for Flexible Job Shop Scheduling Problem with Tree-Structure Job Precedence Constraints." *Applied Soft Computing* 125:109235. <https://doi.org/10.1016/j.asoc.2022.109235>.
- Zuo, W., and Y. Gao. 2025. "Solving Numerical and Engineering Optimization Problems Using a Dynamic Dual-Population Differential Evolution Algorithm." *International Journal of Machine Learning and Cybernetics* 16 (3): 1701–1760. <https://doi.org/10.1007/s13042-024-02361-7>.